

UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II

SCUOLA POLITECNICA E DELLE SCIENZE DI BASE

DIPARTIMENTO DI INGEGNERIA ELETTRICA E TECNOLOGIE
DELL'INFORMAZIONE

CORSO DI LAUREA TRIENNALE IN INFORMATICA

A FANCY L^AT_EX TEMPLATE FOR YOUR THESIS
AT UNINA

Relatori

Megadir. Prof. Guidobaldo Maria

RICCARDELLI

Duca Conte Francesco BARAMBANI

Baronessa Filiguelli DE BONCHAMPS

Correlatore

Megadir. Clam. Duca Conte Pier

Carlo ing. SEMENZARA

Candidato

Marco FESTA

N86004979

Anno Accademico 2025–2026

Sommario

Il presente lavoro di tesi illustra le attività di ricerca e sviluppo condotte durante il periodo di tirocinio presso **InfoCube**, focalizzandosi sull'estrazione, strutturazione e interrogazione semantica avanzata di documenti legali e normativi. Il progetto si inserisce nel contesto più ampio di **LUDOVICA** (*Language Understanding for Document Optimization and Validation In Civil Administration*), un motore generativo basato sull'Intelligenza Artificiale volto a supportare la pubblica amministrazione locale nell'elaborazione e stesura di atti amministrativi.

Il problema tecnico principale affrontato in questo lavoro è la trasformazione di documenti PDF complessi in una base di conoscenza solida e interrogabile dalle macchine (**Knowledge Graph**). A tale scopo, la rigida gerarchia tipica degli atti normativi (Titoli, Capi, Articoli, Commi) è stata modellata e salvata all'interno del database a grafo **Neo4j**[1], permettendo di preservare fedelmente le relazioni strutturali e concettuali dei testi originali.

Per abilitare il recupero intelligente e contestuale delle informazioni, è stato implementato un sistema di **Semantic Search**. Questa fase ha incluso il confronto di molteplici **modelli di embedding**, con l'obiettivo di osservarne il comportamento sul linguaggio giuridico italiano. Le scelte architetturali sono state valutate mediante un benchmark diagnostico di domande annotate, confrontando un approccio di ricerca vettoriale classica, basato sulla *cosine similarity*, con una strategia che impiega le relazioni giuridiche del grafo per riordinare i risultati.

Sul benchmark diagnostico di 70 domande, la strategia arricchita dalla struttura aumenta la completezza soprattutto nei casi in cui la risposta coinvolge disposizioni collegate, mentre a cutoff fisso introduce un costo nei casi puramente semantici. Il confronto con un encoder contestuale mostra inoltre che il *late chunking* non rende ridondante il contributo delle relazioni esplicite nelle condizioni valutate. I risultati descrivono pertanto un beneficio condizionato alla tipologia della domanda e non costituiscono una stima delle prestazioni su traffico reale.

Indice

1	Introduzione	1
1.1	Obiettivi del lavoro di tesi	2
1.2	Confronto con tecniche esistenti	2
1.3	Struttura della tesi	3
2	Tecnologie utilizzate	5
2.1	Linguaggi di Programmazione	5
2.2	Strumenti per l'estrazione del Testo	5
2.3	Scelta del Database	6
2.4	LLM e Modelli di Embedding Semantici	7
2.5	Deployment e Containerizzazione	8
3	Architettura del Sistema	9
3.1	Vista generale dell'architettura	9
3.2	Architettura della pipeline di acquisizione e indicizzazione	10
3.2.1	Acquisizione ed estrazione del testo	10
3.2.2	Strutturazione gerarchica e relazioni legali	11
3.2.3	Persistenza nel Knowledge Graph	11
3.2.4	Generazione e memorizzazione degli embedding	11
3.3	Modello informativo e flusso dei dati	12
3.4	Architettura della pipeline di interrogazione	12
3.4.1	Ricerca vettoriale	13
3.4.2	Ricerca arricchita dalla struttura	13
3.5	Deployment e comunicazione tra i componenti	13
4	Implementazione Software	14
4.1	Estrazione dei Documenti	14
4.1.1	Open Data Loader e la Gestione dei Tagged PDF	14
4.1.2	Docling e l'Integrazione dei Motori OCR	15
4.1.3	Progettazione del Custom Triage Processor	16
4.2	Strutturazione dei Documenti tramite LLM	19
4.2.1	Il Modello e la Strategia di Generazione	19
4.2.2	Estrazione delle Relazioni Semantiche	20

4.3	Importazione dati nei Database	21
4.3.1	I Modelli di Dominio	21
4.3.2	Interfaccia per il disaccoppiamento della persistenza	22
4.3.3	Architettura del Database Neo4j	23
4.3.4	Dettagli Implementativi dell'Ingestion su Neo4j	24
4.3.5	Risultati e Metriche	25
4.3.6	Visualizzazione del Grafo	27
4.4	Creazione degli Embedding	28
4.4.1	Differenza nell'Arricchimento Contestuale: Standard vs Late Chunksing	28
5	Ricerca Semantica	30
5.1	Le due strategie e che cosa isolano	30
5.2	S1: la ricerca vettoriale pura	31
5.3	S2: la ricerca arricchita dalla struttura	32
5.3.1	Step 1: il bacino vettoriale	32
5.3.2	Step 2: i semi del ramo giuridico	32
5.3.3	Step 3: il ramo giuridico	32
5.3.4	Step 4: la fusione RRF pesata	33
5.3.5	Step 5: la testa riservata alla vettoriale	34
5.3.6	Perché il peso giuridico resta 3	34
5.3.7	Un ramo rimosso: l'adiacenza fra commi	35
6	Valutazione Sperimentale	36
6.1	Il banco di domande	36
6.1.1	Struttura del banco	36
6.1.2	Etichette primarie e di supporto	39
6.2	Le metriche	40
6.2.1	Configurazione sperimentale	40
6.3	Risultati sul benchmark diagnostico	41
6.3.1	Il guadagno dipende dalla categoria e non segue interamente l'attesa iniziale	42
6.3.2	Il segno del risultato si ripete sui tre encoder	43
6.4	Il costo della struttura	43
6.4.1	Il costo di ordinamento	43
6.4.2	Il costo computazionale	43
6.5	Il contributo del grafo con un encoder contestuale	44
6.6	Limiti della valutazione	46
6.7	Sintesi del capitolo	47
	Bibliografia	48

Elenco delle figure

3.1	Architettura complessiva del sistema.	10
4.1	Flowchart del Custom Triage Processor	18
4.2	Class Diagram dei modelli di dominio	22
4.3	Rappresentazione visiva delle relazioni instaurate tra i nodi dell'articolo 8 di un atto normativo.	25
4.4	Rappresentazione visiva di un estratto del database a grafo Neo4j.	27

Elenco delle tabelle

4.1	Statistica dei nodi inseriti nel database Neo4j.	26
4.2	Statistica delle relazioni inserite nel database Neo4j.	26
5.1	Moltiplicatori per tipo di relazione giuridica nel ramo a grafo.	33
5.2	Comportamento della fusione RRF pesata con $k_{\text{RRF}} = 60$ e pesi $4/3$	33
6.1	Composizione del benchmark diagnostico.	37
6.2	Confronto complessivo fra ricerca vettoriale pura (S1) e ricerca arricchita dalla struttura (S2), su 70 domande e tre modelli di embedding.	41
6.3	Complete HIT@10 per categoria.	42
6.4	Complete HIT@10 per modello di embedding, sul banco principale.	43
6.5	Latenza di <code>strategy.search()</code> , mediata sui tre modelli. Esclude la codifica della domanda.	44
6.6	Contributo della struttura mantenendo fisso l'encoder contestuale.	44
6.7	Complete HIT@10 per categoria mantenendo fisso l'encoder contestuale.	45
6.8	Confronto fra S1 e S2 sulle dieci domande MULTI_PARAGRAPH, mantenendo fisso l'encoder contestuale.	45

–1–

Introduzione

CONTENTS: 1.1 Obiettivi del lavoro di tesi. 1.2 Confronto con tecniche esistenti. 1.3 Struttura della tesi.

La pubblica amministrazione locale italiana fa quotidianamente affidamento su una vasta e complessa mole di testi normativi, atti e delibere. L’elaborazione automatica di tali documenti è tuttavia ostacolata da diverse sfide intrinseche: i testi sono spesso archiviati in formati non strutturati (come i PDF), presentano difformità di impaginazione, impiegano un linguaggio burocratico peculiare e si articolano in una rigida e profonda struttura gerarchica. Quando si tenta di automatizzare la ricerca all’interno di tali documenti mediante sistemi basati su Intelligenza Artificiale Generativa, un approccio fondato esclusivamente sul recupero di frammenti testuali tramite similarità semantica può non rappresentare esplicitamente la struttura gerarchica del documento e le relazioni esistenti tra le diverse disposizioni. Nel dominio giuridico, dove la corretta individuazione del contesto costituisce una componente centrale del processo di retrieval, tale aspetto assume particolare rilevanza [2]. È in questo scenario che si inserisce il lavoro di tesi, svolto nell’ambito delle attività di tirocinio presso **InfoCube** per contribuire allo sviluppo del progetto **LUDOVICA**, il cui obiettivo finale è fornire ai funzionari della PA un motore generativo affidabile, capace di recuperare gli atti pregressi pertinenti e generare template documentali pronti all’uso. Affinché le risposte del sistema siano rigorose e verificabili è emersa l’esigenza di adottare un’architettura dati che introduca una solida topologia informativa.

Il problema tecnico fondamentale affrontato consiste quindi nella necessità di **convertire i documenti normativi complessi in una base di conoscenza interrogabile** (*Knowledge Graph*) che conservi fedelmente la struttura semantica e gerarchica originale. In letteratura, i Knowledge Graph sono impiegati per rappresentare entità e relazioni in forma esplicita, rendendo interrogabili collezioni di conoscenza eterogenee e interconnesse [3]. Nel dominio giuridico, inoltre, standard come Akoma Ntoso formalizzano la natura gerarchica dei documenti legislativi e amministrativi e prevedono meccanismi espliciti per citazioni e riferimenti incrociati [4].

1.1 Obiettivi del lavoro di tesi

Al fine di superare i limiti esposti e dotare il sistema di solidi meccanismi per l'interpretazione del linguaggio giuridico, il lavoro si è articolato nei seguenti obiettivi:

1. **Modellazione topologica e digitalizzazione:** Costruire una pipeline di inserimento per tradurre documenti legali non strutturati in un database a grafo (*Neo4j*), definendo rigorosamente la mappatura delle relazioni.
2. **Semantic Search per il dominio giuridico:** Sviluppare il modulo di interrogazione intelligente del database e confrontare il comportamento di più modelli di embedding nel recupero di disposizioni normative in lingua italiana.
3. **Analisi degli approcci di Retrieval:** Misurare, mediante un benchmark diagnostico, in quali condizioni una strategia che impiega le relazioni giuridiche del grafo migliori o peggiori il recupero rispetto a una baseline puramente vettoriale basata sulla *cosine similarity*.

Per raggiungere gli obiettivi prefissati, è stata ideata un'architettura di elaborazione in grado di trasformare documenti non strutturati in una rete di nodi interconnessi. L'approccio proposto si articola in una pipeline a più fasi: inizialmente, i documenti originali (principalmente in formato PDF) vengono processati tramite strumenti di parsing avanzati e OCR per estrarne accuratamente il testo e convertirlo in un formato intermedio (Markdown). Successivamente, un *Large Language Model* analizza tale testo per riconoscerne la suddivisione gerarchica (Titoli, Capi, Articoli, Commi) e le relazioni intrinseche tipiche della struttura giuridica (ad esempio un comma che cita un altro comma). Questi elementi vengono strutturati in formato JSON e iniettati all'interno del grafo, in cui ogni entità diventa un nodo e l'appartenenza a una data sezione diventa una relazione. Successivamente, i contenuti testuali vengono vettorializzati e salvati sotto forma di *embedding*. In fase di interrogazione, il sistema affianca alla similarità semantica la navigazione esplicita delle relazioni del grafo, con l'obiettivo di promuovere disposizioni collegate che completano il contesto recuperato. Il presente lavoro valuta il recupero dei commi e il loro ordinamento; la qualità di un'eventuale risposta generata a valle rimane fuori dal perimetro sperimentale.

1.2 Confronto con tecniche esistenti

Le architetture di *Retrieval-Augmented Generation* (RAG) combinano le capacità generative dei *Large Language Models* con il recupero di informazioni provenienti da una base di conoscenza esterna [5]. Nelle implementazioni basate su retrieval vettoriale, i documenti vengono generalmente suddivisi in porzioni testuali, rappresentate mediante embedding e recuperate sulla base della similarità semantica rispetto alla query dell'utente.

L'efficacia di questo processo dipende tuttavia non soltanto dal modello generativo, ma anche dalla qualità delle informazioni recuperate, tale aspetto assume particolare

rilevanza nel dominio giuridico. Pipitone e Houir Alami. [2], attraverso il benchmark *LegalBench-RAG*, evidenziano l'importanza di recuperare segmenti testuali minimi e altamente pertinenti, mostrando come il recupero di porzioni di testo troppo ampie o imprecise possa introdurre informazioni non necessarie nel contesto fornito al modello.

Un ulteriore limite deriva dal fatto che la suddivisione di un documento in frammenti indipendenti tende a non rappresentarne esplicitamente l'organizzazione strutturale. Questo aspetto è particolarmente rilevante per i testi normativi, nei quali una disposizione non costituisce necessariamente un'unità isolata, ma appartiene a una gerarchia composta, ad esempio, da titoli, capi, articoli e commi e può contenere riferimenti ad altre disposizioni. Il solo valore di similarità tra la query e il contenuto testuale consente quindi di stimare la pertinenza semantica di un frammento, ma non codifica direttamente tali relazioni strutturali.

Per affrontare problemi di questo tipo sono stati proposti approcci che affiancano al retrieval vettoriale rappresentazioni strutturate della conoscenza. Edge et al. [6] propongono un approccio *GraphRAG* nel quale, a partire dai documenti del corpus, viene costruito un Knowledge Graph di entità e relazioni. Il grafo viene successivamente organizzato in entità e relazioni, in questo modo, il sistema può recuperare informazioni distribuite in differenti parti del corpus e affrontare interrogazioni che richiedono una visione più globale dei documenti.

Alla luce di tali considerazioni, il sistema sviluppato in questa tesi conserva esplicitamente la struttura nativa degli atti normativi e alcune delle relazioni presenti tra i relativi elementi, affiancandole alla rappresentazione vettoriale del contenuto testuale. L'ipotesi alla base del lavoro è che tali informazioni strutturali possano contribuire al recupero del contesto normativo rilevante. La superiorità dell'approccio basato sul grafo non viene pertanto assunta a priori, ma viene valutata sperimentalmente attraverso il confronto con una baseline esclusivamente vettoriale.

1.3 Struttura della tesi

Il presente elaborato è organizzato in sei capitoli, di seguito brevemente introdotti.

Il Capitolo 2 — Tecnologie utilizzate fornisce una panoramica dello stack tecnologico adottato: dai linguaggi di programmazione agli strumenti di parsing e OCR per l'estrazione del testo, passando per il database a grafo, fino ai modelli di linguaggio e di embedding semantico impiegati per la strutturazione e la vettorializzazione dei contenuti normativi.

Il Capitolo 3 — Architettura del Sistema illustra l'architettura complessiva del sistema, descrivendo i componenti principali e il flusso di dati attraverso le diverse fasi della pipeline: dall'acquisizione dei documenti grezzi fino all'interrogazione da parte degli utenti finali.

Il Capitolo 4 — Implementazione Software approfondisce nel dettaglio le singole fasi della pipeline di elaborazione: l'estrazione del testo dai documenti PDF, la struttura-

zione gerarchica operata dai Large Language Models, l'ingestion dei dati strutturati su Neo4j e la creazione degli embedding per la ricerca semantica.

Il **Capitolo 5 – Ricerca Semantica** definisce le due strategie di recupero confrontate nel lavoro: la baseline puramente vettoriale e la strategia ibrida che usa le relazioni giuridiche come segnale di riordinamento. Ne descrive inoltre i componenti, i parametri e le scelte progettuali.

Il **Capitolo 6 – Valutazione Sperimentale** presenta il benchmark diagnostico, le metriche e i risultati del confronto, distinguendo i casi in cui la struttura migliora la completezza del recupero da quelli in cui introduce un costo di ordinamento o di selezione.

–2–

Tecnologie utilizzate

Per far fronte alle sfide di estrazione, elaborazione, archiviazione e interrogazione dei documenti normativi, è stato impiegato uno stack tecnologico che combina strumenti di parsing avanzato, modelli di LLM ed embedding e database specializzati. Di seguito vengono presentate le tecnologie adottate, illustrando per ciascuna le motivazioni che ne hanno guidato la scelta.

2.1 Linguaggi di Programmazione

- **Python:** È stato il linguaggio cardine per la costruzione dell'intera pipeline del progetto. La scelta è ricaduta su Python per diverse ragioni:
 - **Ecosistema dedicato ai modelli linguistici:** Python dispone di librerie mature e ampiamente supportate (come `sentence-transformers`) che hanno permesso di integrare agevolmente i modelli di embedding senza dover reimplementare componenti di basso livello.
 - **Ottima integrazione con tutte le librerie utilizzate:** Python risulta perfettamente integrato con tutti gli strumenti e le librerie adottate nell'intero stack tecnologico (dai tool di estrazione testuale ai modelli LLM e al database Neo4j tramite API native), permettendo di orchestrare l'intera pipeline in modo fluido e senza la necessità di complessi adattatori o wrapper di terze parti.

2.2 Strumenti per l'estrazione del Testo

La fase di estrazione ha dovuto confrontarsi con documenti PDF eterogenei: alcuni nativamente digitali e ben strutturati, altri derivanti da scansioni cartacee di qualità variabile. Per questo motivo è stata adottata una combinazione di strumenti complementari.

- **Open Data Loader**[7]: Libreria open-source per l'estrazione e il caricamento di dati da fonti eterogenee. È stata selezionata per due caratteristiche fondamentali:
 - **Determinismo**: A differenza di approcci basati su modelli probabilistici, Open Data Loader estrae il testo in modo deterministico, garantendo coerenza dei risultati, un requisito indispensabile nel dominio legale, dove ogni modifica del testo estratto tra un'esecuzione e l'altra minerebbe la fiducia nel sistema.
 - **Conversione diretta in Markdown**: Il Markdown si rivela un formato testuale ottimale per i LLM. La sua sintassi leggera preserva fedelmente la struttura logica del documento originale, fornendo al modello un contesto già strutturato che riduce significativamente gli errori nella successiva fase di parsing gerarchico.
- **Docling**[8]: Libreria specializzata nell'analisi del layout documentale e nell'estrazione di contenuti strutturati. È stata integrata come supporto complementare a Open Data Loader per la gestione di PDF più complessi: supporta documenti multi-colonna, tabelle e layout non lineari, frequenti negli atti della pubblica amministrazione.
- **RapidOCR**[9]: Motore di *Optical Character Recognition* impiegato per digitalizzare i documenti derivanti da scansioni visive. La scelta è stata guidata da un'analisi comparativa tra diversi motori OCR:
 1. **EasyOCR**: Configurato di default all'interno di Docling, adotta modelli basati su Deep Learning altamente accurati ma estremamente esigenti in termini computazionali. Anche isolando l'elaborazione sulla singola pagina, il consumo di RAM era così elevato che rendeva impossibile l'utilizzo nell'ambiente di produzione containerizzato.
 2. **Tesseract OCR**: Si è dimostrato estremamente leggero e pienamente compatibile con i limiti di memoria imposti, scongiurando qualsiasi crash del container. Tuttavia, nel contesto specifico del linguaggio legale italiano, il motore non è stato in grado di interpretare correttamente il layout, fallendo l'estrazione del testo e restituendo output incompleti o disallineati rispetto alla struttura originale del documento.
 3. **RapidOCR**: Questo specifico modulo è stato infine selezionato per l'estrazione del testo dalle scansioni visive. RapidOCR si è rivelato il compromesso ideale, offrendo un perfetto bilanciamento tra l'accuratezza del riconoscimento del testo italiano e il consumo di RAM del container.

2.3 Scelta del Database

La duplice natura del problema — da un lato la necessità di rappresentare relazioni gerarchiche complesse, dall'altro l'esigenza di una ricerca semantica efficiente — ha

portato all'adozione di un database a grafo specializzato. La scelta è ricaduta su Neo4j, il database a grafo più diffuso e maturo sul mercato, per le seguenti motivazioni:

- **Modellazione naturale della gerarchia normativa:** La struttura di un atto normativo è intrinsecamente un grafo, in cui ogni elemento è un nodo e le relazioni di appartenenza sono archi. Neo4j consente di mappare questa topologia in modo diretto, senza le complessità delle JOIN ricorsive tipiche dei database relazionali.
- **Supporto nativo per la Vector Search:** Il database permette di memorizzare rappresentazioni vettoriali come proprietà dei nodi e di creare indici dedicati, configurabili con funzioni di similarità coseno o euclidea. Nelle versioni recenti di Neo4j, tali indici supportano vettori con dimensionalità compresa tra 1 e 4096 [10]. Questa funzionalità risulta fondamentale per le operazioni di ricerca basate sulla *cosine similarity*.

Per interagire con il database e gestire le operazioni di interrogazione e popolamento del grafo, è stato adottato **Cypher** [11], il linguaggio dichiarativo nativo dell'ecosistema Neo4j.

2.4 LLM e Modelli di Embedding Semantici

Il cuore intelligente del sistema risiede nei modelli di linguaggio e di embedding, responsabili rispettivamente della strutturazione semantica dei documenti e della loro vettorializzazione per la ricerca.

- **Gemini 3.5 Flash**[12]: *Large Language Model* di Google, utilizzato tramite API per il compito cruciale della strutturazione dei documenti normativi. Le motivazioni alla base di questa scelta sono:
 - **Aderenza rigorosa allo schema JSON:** Gemini 3.5 Flash supporta la generazione di output strutturati conformi a uno schema JSON predefinito [13]. Nell'ambito del progetto, questa funzionalità ha consentito di rappresentare la gerarchia normativa (Titoli, Capi, Articoli, Commi) in modo consistente e automaticamente processabile dalle fasi successive della pipeline.
 - **Comprensione della lingua italiana:** Il modello ha mostrato una solida competenza nell'elaborazione del linguaggio burocratico italiano, riconoscendo correttamente la terminologia giuridica e le espressioni formulari tipiche degli atti amministrativi.
 - **Rapporto costo/qualità:** Rispetto a modelli più potenti ma significativamente più costosi (come GPT-4 o Claude), Gemini 3.5 Flash ha offerto un compromesso ottimale tra qualità dell'output strutturato e costo per token, aspetto rilevante considerando l'elaborazione batch di centinaia di documenti.

- **Modelli di Embedding:** La vettorializzazione del testo normativo rappresenta un elemento critico per la qualità della ricerca semantica. Sono stati confrontati molteplici modelli per osservarne il comportamento sul dominio giuridico italiano, distinguendo la completezza del recupero dalla qualità dell'ordinamento anziché assumere l'esistenza di un vincitore unico. I test più approfonditi hanno coinvolto i seguenti modelli:
 - **google/embeddinggemma-300m:** Modello sviluppato da Google che trasforma il testo in vettori a 768 dimensioni. È costruito sull'architettura Gemma 3 ed è addestrato su oltre 100 lingue. Mediante la tecnica *Matryoshka Representation Learning*, la rappresentazione può essere ridotta a 512, 256 o 128 dimensioni, introducendo un compromesso tra compattezza e qualità dell'embedding [14].
 - **perplexity-ai/pplx-embed-v1-0.6b:** Modello sviluppato da Perplexity AI che produce vettori a 1024 dimensioni ed è progettato per compiti di retrieval multilingue, tra cui la ricerca semantica e i sistemi RAG [15].
 - **perplexity-ai/pplx-embed-context-v1-0.6b:** Variante contestuale del modello Perplexity, anch'essa produce vettori a 1024 dimensioni. Durante la rappresentazione dei singoli chunk incorpora informazioni provenienti dal contesto globale del documento, anziché trattare ciascun frammento in modo completamente isolato [15]. Il modello applica il principio del *Late Chunking* [16], con l'obiettivo di preservare nelle rappresentazioni locali una parte dell'informazione disponibile nel documento completo.

2.5 Deployment e Containerizzazione

- **Docker:** L'intero ecosistema è stato modularizzato attraverso lo sviluppo di container isolati e specifici per ciascun componente della pipeline. Questa scelta architetturale è stata dettata da esigenze sia tecniche che operative:
 - **Isolamento delle dipendenze:** Ogni fase della pipeline (estrazione, strutturazione, ingestion, embedding) presenta dipendenze software potenzialmente conflittuali. Docker ha permesso di incapsulare ciascuna in un ambiente dedicato, eliminando i conflitti di versione tra librerie e garantendo la riproducibilità dell'ambiente di esecuzione su qualsiasi macchina.
 - **Modularità e manutenibilità:** La separazione in container indipendenti ha permesso di sviluppare, testare e aggiornare ciascun modulo della pipeline in isolamento, semplificando la manutenzione.
 - **Scalabilità orizzontale:** L'architettura a container costituisce un prerequisito naturale per futuri scenari di deployment distribuito in cui diverse istanze della pipeline possano operare concorrentemente su lotti distinti di documenti.

–3–

Architettura del Sistema

CONTENTS: 3.1 Vista generale dell’architettura. 3.2 Architettura della pipeline di acquisizione e indicizzazione. 3.3 Modello informativo e flusso dei dati. 3.4 Architettura della pipeline di interrogazione. 3.5 Deployment e comunicazione tra i componenti.

Il sistema sviluppato ha l’obiettivo di trasformare una collezione di documenti normativi non strutturati in una base di conoscenza interrogabile. A tal fine, l’architettura integra tecniche di estrazione documentale, modelli linguistici, rappresentazioni vettoriali e un database a grafo.

Il presente capitolo descrive l’organizzazione complessiva del sistema, le responsabilità assegnate ai suoi componenti e il percorso seguito dai dati. I dettagli relativi alle singole librerie, alle classi software e alle procedure di importazione sono invece approfonditi nel Capitolo 4. La valutazione sperimentale delle strategie di ricerca è demandata ai capitoli successivi.

3.1 Vista generale dell’architettura

La Figura 3.1 mostra i componenti principali e le trasformazioni attraversate dai dati. Sebbene il percorso di acquisizione proceda prevalentemente in modo sequenziale, l’architettura complessiva è suddivisa in due macroprocessi:

1. una **pipeline offline di acquisizione e indicizzazione**, eseguita per trasformare i documenti originali e popolare la base di conoscenza;
2. una **pipeline online di interrogazione**, attivata quando una domanda in linguaggio naturale deve essere confrontata con il corpus indicizzato.

Il database Neo4j costituisce il punto di integrazione tra i due macroprocessi. Durante l’acquisizione riceve nodi, relazioni ed embedding; durante l’interrogazione fornisce

invece i candidati semanticamente pertinenti e il contesto strutturale necessario per arricchirli.

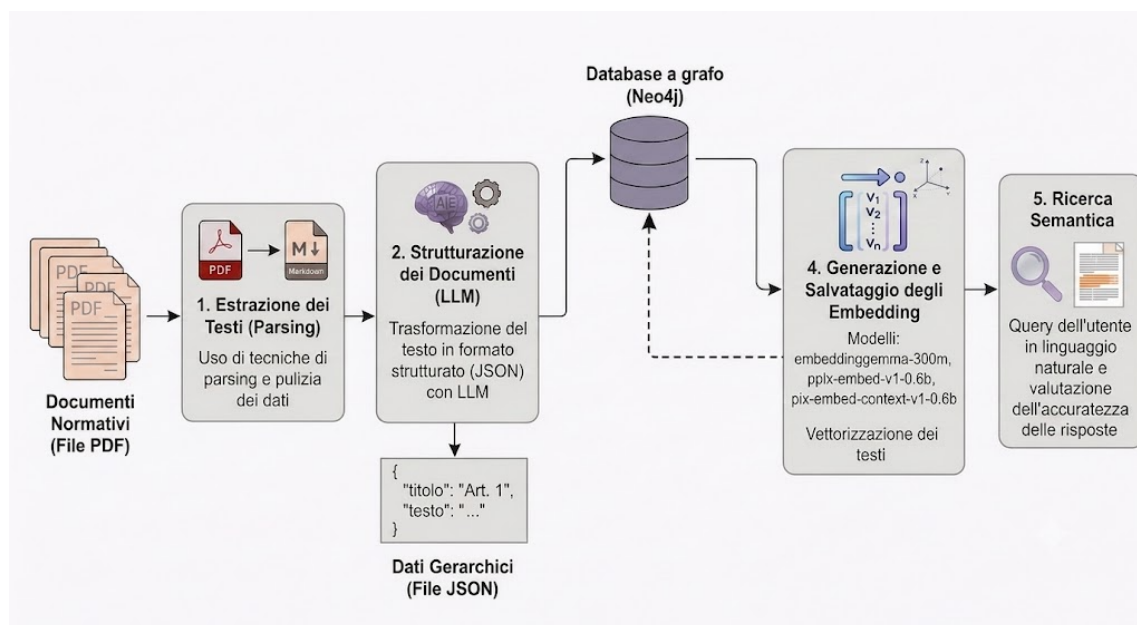


Figura 3.1: Architettura complessiva del sistema.

La separazione tra i due flussi evita che le operazioni costose di parsing, strutturazione e vettorizzazione debbano essere ripetute per ogni query. Tali attività vengono svolte in anticipo, mentre in fase di ricerca si eseguono soltanto le operazioni necessarie al recupero delle informazioni.

3.2 Architettura della pipeline di acquisizione e indicizzazione

La pipeline offline comprende cinque responsabilità principali: acquisizione dei documenti, estrazione del testo, strutturazione, persistenza e indicizzazione vettoriale.

3.2.1 Acquisizione ed estrazione del testo

Il punto di ingresso è costituito dai documenti normativi in formato PDF. Un componente di triage ne analizza le caratteristiche e seleziona la strategia di estrazione più appropriata. I documenti digitali strutturati possono essere elaborati sfruttando direttamente le informazioni logiche incorporate nel file; quelli con layout complesso richiedono un'analisi più articolata; per le pagine acquisite come immagini è invece necessario il riconoscimento ottico dei caratteri.

L'output comune di questa fase è un documento in formato Markdown. La scelta di un formato intermedio testuale separa l'acquisizione fisica del PDF dalla successiva interpretazione logica: i componenti a valle non devono conoscere il parser utilizzato e ricevono sempre una rappresentazione uniforme. Il Markdown conserva inoltre elementi utili, quali titoli, paragrafi ed elenchi, senza introdurre la complessità del formato sorgente.

3.2.2 Strutturazione gerarchica e relazioni legali

Il testo estratto viene sottoposto a un modello linguistico con il compito di ricostruire l'organizzazione dell'atto. Questa elaborazione è articolata in due passaggi. Nel primo vengono riconosciuti i metadati e la gerarchia composta da titoli, capi, articoli e commi; nel secondo vengono individuati i collegamenti giuridico-logici tra le disposizioni, come citazioni, deroghe, interpretazioni, abrogazioni e sostituzioni.

Il risultato viene rappresentato mediante file JSON conformi a uno schema prestabilito. Il JSON assume quindi il ruolo di contratto tra il modulo di strutturazione e il modulo di persistenza.

3.2.3 Persistenza nel Knowledge Graph

I dati strutturati vengono trasformati in entità del dominio e importati in Neo4j. Il grafo conserva tre categorie di informazione:

- la **gerarchia documentale**, che collega l'ente emittente all'atto e quest'ultimo a titoli, capi, articoli e commi;
- la **sequenza testuale**, che rappresenta l'ordine dei commi appartenenti allo stesso articolo;
- le **relazioni legali**, che collegano disposizioni semanticamente o giuridicamente dipendenti.

Il comma costituisce l'unità atomica scelta per il recupero dell'informazione. Tale granularità permette di restituire porzioni circoscritte del testo senza perdere la possibilità di risalire ai contenitori gerarchici o di raggiungere disposizioni collegate. Ogni nodo è inoltre dotato di un identificativo che ne codifica la provenienza e la posizione nel documento, rendendo possibile ricostruire il percorso dall'informazione recuperata alla fonte originale.

3.2.4 Generazione e memorizzazione degli embedding

Dopo il popolamento del grafo, il contenuto dei commi viene convertito in rappresentazioni vettoriali. Gli embedding semantici codificano il significato linguistico del testo e rendono confrontabili le unità normative con le query degli utenti. Per evitare che un comma venga rappresentato come frammento completamente isolato, il processo può includere

informazioni provenienti dalla sua gerarchia oppure dal resto dell'articolo, secondo la strategia supportata dal modello selezionato.

Gli embedding vengono memorizzati come proprietà dei nodi di tipo Paragraph. In questo modo, testo, provenienza, posizione nel documento e rappresentazione vettoriale rimangono associati alla stessa entità.

La memorizzazione nel medesimo database consente di passare dal risultato di una ricerca semantica alla navigazione delle relazioni con un'unica infrastruttura di persistenza.

3.3 Modello informativo e flusso dei dati

Nel corso della pipeline, lo stesso documento attraversa rappresentazioni differenti, ciascuna delle quali risponde a una responsabilità specifica:

PDF → Markdown → JSON → grafo → indicizzazione semantica

Il PDF costituisce la fonte originale; il Markdown rappresenta il testo normalizzato; il JSON descrive in forma esplicita le entità e le relazioni riconosciute; il grafo rende tali informazioni persistenti e navigabili; gli embedding aggiungono infine una rappresentazione numerica adatta al recupero semantico.

Nel modello concettuale, un ente emette uno o più atti normativi. Ciascun atto contiene una gerarchia che può comprendere titoli, capi, articoli e commi. Poiché non tutti i documenti presentano ogni livello, il modello ammette percorsi nei quali alcuni contenitori intermedi sono assenti. Le relazioni trasversali trasformano questa struttura, altrimenti puramente arborea, in un grafo: un comma può essere connesso al successivo oppure richiamare una disposizione collocata in un altro punto dell'atto.

Questa rappresentazione permette di mantenere distinte tre nozioni di prossimità:

- la **prossimità semantica**, misurata nello spazio degli embedding testuali;
- la **prossimità gerarchica e sequenziale**, determinata dalla collocazione degli elementi nel documento;
- la **prossimità giuridica**, espressa dai riferimenti espliciti tra disposizioni.

La distinzione è rilevante perché due commi possono risultare semanticamente simili pur appartenendo a contesti differenti oppure, al contrario, essere lessicalmente diversi ma strettamente collegati da una relazione normativa.

3.4 Architettura della pipeline di interrogazione

La pipeline online riceve una domanda in linguaggio naturale e produce una graduatoria di unità normative pertinenti. Per garantire un confronto controllato, le strategie di ricerca condividono il corpus, l'unità di indicizzazione e la rappresentazione della query; differiscono invece per l'impiego delle informazioni topologiche.

3.4.1 Ricerca vettoriale

Nella configurazione di base, la query viene trasformata in un embedding mediante lo stesso modello utilizzato per indicizzare i commi. Il vettore ottenuto viene confrontato con quelli memorizzati nel database e i nodi vengono ordinati sulla base della similarità coseno. Il risultato è costituito dai primi k commi della graduatoria.

Questa strategia utilizza esclusivamente la vicinanza nello spazio semantico durante la selezione dei risultati e rappresenta pertanto la baseline rispetto alla quale valutare l'apporto della struttura. La presenza del grafo come supporto di memorizzazione non implica, in questo caso, che le sue relazioni contribuiscano al ranking.

3.4.2 Ricerca arricchita dalla struttura

La seconda configurazione parte ugualmente dalla query e dal recupero di candidati semanticamente pertinenti, ma integra le informazioni contenute nella topologia. A partire dai nodi individuati è possibile considerare il contesto gerarchico, i commi adiacenti e le disposizioni collegate da relazioni legali.

Il grafo non sostituisce quindi la ricerca semantica, ma introduce segnali ulteriori con cui arricchire il recupero. Questa separazione consente di attribuire le eventuali differenze osservate all'impiego della struttura, mantenendo invariati gli altri elementi del processo sperimentale. La definizione puntuale degli algoritmi, dei parametri di combinazione e delle metriche di valutazione viene presentata nei capitoli dedicati al benchmark.

3.5 Deployment e comunicazione tra i componenti

I moduli principali sono eseguiti in container Docker distinti. Tale organizzazione isola dipendenze eterogenee, quali runtime Java, librerie Python, motori OCR e componenti di elaborazione vettoriale, e permette di aggiornare una fase senza modificare l'ambiente delle altre.

La comunicazione tra le fasi offline avviene attraverso gli artefatti prodotti dalla pipeline e mediante il database condiviso. Markdown e JSON costituiscono punti di disaccoppiamento espliciti, mentre Neo4j rappresenta l'interfaccia persistente tra indicizzazione e interrogazione. La separazione in container non implica necessariamente un'esecuzione distribuita, ma predispone il sistema a elaborare in futuro lotti distinti di documenti mediante istanze indipendenti.

–4–

Implementazione Software

CONTENTS: 4.1 Estrazione dei Documenti. 4.2 Strutturazione dei Documenti tramite LLM. 4.3 Importazione dati nei Database. 4.4 Creazione degli Embedding.

In questo capitolo verranno analizzate nel dettaglio le singole fasi della pipeline di elaborazione, partendo dalle strategie di estrazione del testo grezzo, passando per la strutturazione operata dai Large Language Models, fino ad arrivare alle logiche di ingestion e interrogazione sui database

4.1 Estrazione dei Documenti

La fase iniziale della pipeline prevede l'estrazione del testo dai documenti normativi in formato PDF.

Nelle prime fasi di sviluppo del progetto, l'intero documento PDF veniva passato direttamente al Large Language Model, demandando a quest'ultimo sia il compito di estrazione del testo grezzo sia quello di strutturazione sintattica. Questo approccio presentava forti criticità, esponeva il sistema al rischio di allucinazioni ed era inefficiente nel parsing di strutture complesse. Per superare tali limitazioni, la pipeline è stata modificata separando nettamente l'estrazione del testo dalla sua successiva strutturazione.

4.1.1 Open Data Loader e la Gestione dei Tagged PDF

Per l'estrazione del testo da documenti digitali nativi è stata adottata la libreria open-source *Open Data Loader*. Questo strumento converte localmente i file PDF in formati ottimizzati per i LLM, specificamente Markdown. Il vantaggio di questa soluzione risiede nella sua natura deterministica: a parità di input viene garantito il medesimo output, azzerando le variabilità tipiche dei modelli generativi.

Durante l'analisi dei documenti legali, una delle criticità maggiori riguarda i ritorni a capo spuri, generati per ragioni puramente stilistiche (ad esempio, il raggiungimento del

marginale della pagina). Se non gestiti correttamente, questi si traducono in doppi ritorni a capo nel testo Markdown finale, frammentando artificialmente i periodi e compromettendo la qualità della successiva fase di chunking. Per risolvere questa criticità alla radice, l'estrazione è stata configurata abilitando il parametro specifico `use_struct_tree=True`.

Quando questo parametro è attivo, la libreria modifica il suo approccio di lettura, verificando innanzitutto se il file analizzato appartiene alla categoria dei *Tagged PDF*. A differenza dei PDF standard, che definiscono esclusivamente il layout bidimensionale e visivo, i Tagged PDF incorporano un livello gerarchico logico e invisibile: il *Logical Structure Tree*. Questo albero organizza il contenuto tramite elementi strutturali del tutto simili ai tag HTML (come `<H1>`, `<P>`, `<Table>`, `<L>`).

Di conseguenza, abilitando `use_struct_tree=True`, *Open Data Loader* è in grado di ignorare le fallaci euristiche geometriche e navigare direttamente la struttura logica del documento. Questo meccanismo permette all'estrattore di mappare ogni porzione di testo discriminando con precisione assoluta i semplici a capo visivi dalle reali interruzioni di paragrafo, restituendo un flusso di testo continuo e fedele all'originale.

Nel contesto della pubblica amministrazione, la formazione di documenti digitali adeguatamente strutturati non costituisce soltanto un'esigenza tecnica. L'articolo 40, comma 1, del Codice dell'Amministrazione Digitale (Decreto Legislativo 7 marzo 2005, n. 82), stabilisce infatti che «le pubbliche amministrazioni formano gli originali dei propri documenti [...] con mezzi informatici» [17].

Le modalità di formazione e gestione di tali documenti sono ulteriormente disciplinate dalle *Linee guida sulla formazione, gestione e conservazione dei documenti informatici*, emanate dall'Agenzia per l'Italia Digitale ai sensi dell'articolo 71 del CAD [18]. In particolare, l'Allegato 2, dedicato ai formati di file, raccomanda di privilegiare formati «aperti, non proprietari, standard de iure, estendibili, parlanti, completamente robusti, indipendenti dal dispositivo» [19]. Tali caratteristiche favoriscono l'interoperabilità, la conservazione nel tempo e il trattamento automatico dei documenti.

Nel caso dei file PDF, una struttura logica correttamente definita consente di rappresentare l'ordine di lettura e il ruolo semantico di elementi quali titoli, paragrafi, elenchi e tabelle. Quando presente, il *Logical Structure Tree* costituisce inoltre per la pipeline sviluppata una fonte di informazioni pre-strutturate, utilizzabile per ricostruire con maggiore precisione la gerarchia e la sequenza logica del contenuto.

4.1.2 Docling e l'Integrazione dei Motori OCR

L'approccio standard basato su *Open Data Loader* mostra evidenti limiti in presenza di documenti non strutturati, con layout complessi o derivanti da scansioni visive, in quanto tale libreria non integra nativamente alcun modulo OCR. Per sopperire a questa mancanza è stata introdotta nel sistema la libreria *Docling*.

A differenza di *Open Data Loader*, *Docling* supporta l'integrazione diretta di motori OCR, configurandosi come un sistema avanzato di parsing e analisi del layout. Questo strumento, specializzato nell'estrazione di metadati, tabelle e testo da formati eterogenei, è stato quindi affiancato a *Open Data Loader* all'interno della pipeline, intervenendo come

supporto specifico per processare tutti quei documenti in cui la sola estrazione con Open Data Loader risulta impossibile o inefficace.

L'introduzione dell'OCR ha sollevato complessità non trascurabili relative alla gestione delle risorse hardware all'interno dell'architettura a container. Il container Docker deputato all'estrazione operava infatti in condizioni in cui non poteva sprecare troppa RAM, date le risorse hardware limitate. Durante i primi test, l'invio massivo di interi documenti scansionati causava la saturazione istantanea della memoria, provocando l'intervento del modulo del kernel Linux (*Out-Of-Memory Killer*) che terminava forzatamente il container.

Per superare il problema del consumo di RAM e prevenire i crash di sistema, è stata implementata una strategia basata sull'elaborazione pagina-per-pagina. Come discusso nel Capitolo 2 relativo alle scelte tecnologiche, a seguito di un'analisi comparativa dei motori supportati da Docling, la scelta è ricaduta su **RapidOCR**. Questo motore ha permesso di mantenere il consumo di memoria adeguato per il funzionamento del container, garantendo l'accuratezza del riconoscimento sul testo legale italiano.

4.1.3 Progettazione del Custom Triage Processor

Il processore di triage nativo di Open Data Loader prevede un flusso standard in cui ogni pagina viene analizzata e smistata: le pagine semplici vengono processate tramite il processo di Open Data Loader (una JVM), mentre le pagine complesse vengono delegate al backend basato su Docling.

Tuttavia, l'applicazione con l'uso del parametro `use_struct_tree=True` introduce un bug logico nel triage di default: i documenti non espressamente *tagged* vengono forzati all'interno del processo standard di Open Data Loader, non venendo mai instradati verso il backend di Docling, escludendo così l'analisi per pagine troppo complesse oppure l'attivazione dell'OCR dove necessario.

Per risolvere questa problematica è stato progettato e implementato un **Custom Triage Processor**. Come illustrato nel diagramma di flusso in **Figura 4.1**, il componente corregge le anomalie di instradamento agendo secondo una logica a tre fasi:

- **Fase 1: Verifica preliminare della struttura.** Il file PDF viene acquisito e il sistema ne analizza i metadati per verificare se è codificato come *Tagged PDF*. In caso affermativo, l'intero documento viene elaborato direttamente da Open Data Loader sfruttando l'albero strutturale nativo. Questo approccio garantisce la massima velocità di esecuzione e una pulizia ottimale del testo estratto.
- **Fase 2: Frazionamento del documento e stima della complessità.** Se il documento risulta privo di struttura (*Non-Tagged*), il flusso prevede la divisione in singole pagine indipendenti. Questa separazione è fondamentale per prevenire sovraccarichi di memoria RAM durante le elaborazioni successive. Per ciascuna pagina, il sistema calcola poi la densità dei caratteri testuali per valutarne il grado di complessità.

- **Fase 3: Triage ed elaborazione mirata della pagina.** Sulla base dell'analisi precedente, ogni singola pagina subisce il processo di smistamento intelligente mostrato nella parte inferiore della Figura 4.1, venendo indirizzata allo strumento di estrazione più adeguato:
 - Se la pagina presenta un layout lineare e un'elevata densità di testo nativo, viene classificata come *Semplice* e affidata al processo di Open Data Loader.
 - Se la pagina contiene elementi geometrici articolati o tabelle annidate, ma il testo rimane comunque selezionabile, viene classificata come *Complessa* e processata tramite Docling Standard (senza abilitare l'OCR).
 - Se la pagina è priva di testo nativo (es. una fotocopia), viene classificata come *Scansione* e inviata al container specializzato di Docling, dove interviene il motore RapidOCR per il riconoscimento ottico dei caratteri.

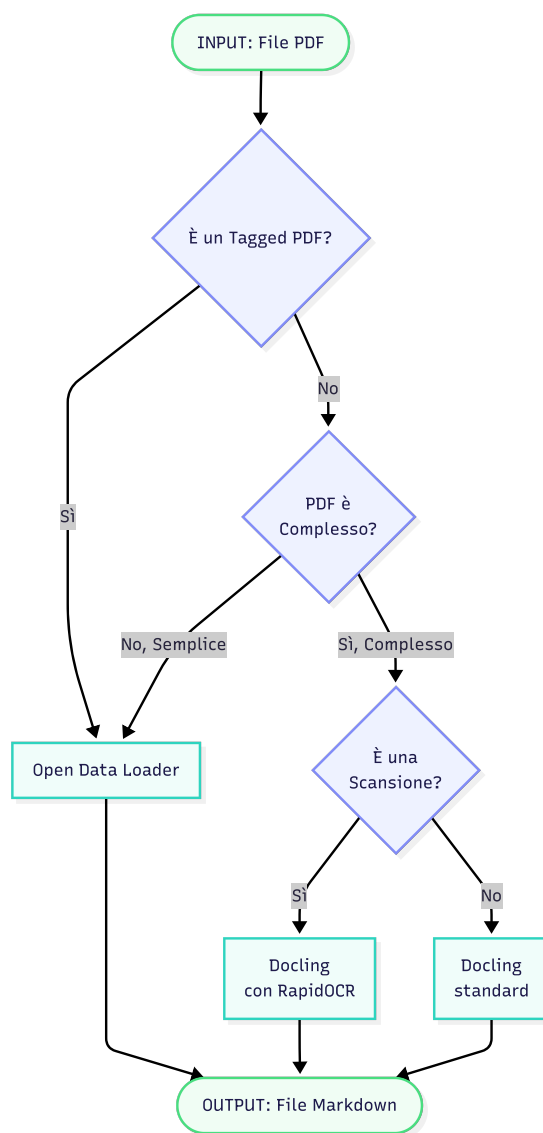


Figura 4.1: Flowchart del Custom Triage Processor

4.2 Strutturazione dei Documenti tramite LLM

Una volta ultimata la fase di estrazione del testo grezzo, illustrata nella sezione precedente, i documenti normativi si presentano sotto forma di file testuali piatti in formato Markdown. Tali file, pur contenendo l'intero testo originale, sono del tutto privi di metadati strutturali e di una formattazione gerarchica chiara. Poiché l'architettura di sistema prevede l'inserimento di questi documenti all'interno di Neo4j risulta indispensabile trasformare il testo non strutturato in una gerarchia rigorosa. L'archiviazione del solo testo grezzo, infatti, risulterebbe del tutto inadeguata e priva di utilità pratica. Una strutturazione accurata rappresenta il prerequisito fondamentale per abilitare qualsiasi operazione avanzata di *information retrieval*, garantendo che i dati possano essere interrogati in modo efficiente e contestuale.

4.2.1 Il Modello e la Strategia di Generazione

Per affrontare il compito cruciale della strutturazione sintattica, è stato impiegato il Large Language Model Gemini 3.5 Flash. A livello di architettura software, il processo è diviso in due step distinti: la generazione di una struttura JSON gerarchica e l'estrazione delle relazioni semantiche tra le entità legali.

L'efficacia e l'accuratezza del processo di strutturazione si fondano su un *System Prompt* ingegnerizzato ad hoc. Il suo compito principale è quello di mappare il contenuto all'interno del seguente schema logico:

- **Metadati (metadata):** Il modello estrae e categorizza i dati identificativi dell'atto. Vengono isolati campi chiave come la data di stipula, il titolo completo e il nome del file originale.
- **Sezioni Introduttive e Conclusive:** Vengono separate dal corpo normativo tutte le sezioni di contorno. Tra queste figurano le premesse iniziali e le epigrafi (preface), gli indici (preamble), le dichiarazioni di chiusura con le relative firme (conclusions) e gli eventuali documenti annessi (attachments).
- **Corpo del Documento (body):** Rappresenta il nucleo dell'algoritmo di strutturazione. Il modello deve nidificare il testo seguendo fedelmente la gerarchia complessa e rigida tipica degli atti normativi italiani: **Titoli > Capi > Articoli > Commi**.

Per garantire la massima coerenza dei dati all'interno del Knowledge Graph, sono state imposte al LLM regole tassative di parsing:

- **Normalizzazione Numerica:** Tutti gli identificativi gerarchici originariamente espressi in numeri romani o in forma testuale (es. "TITOLO I", "CAPO IV") devono essere categoricamente convertiti in numeri interi arabi ("TITOLO 1", "CAPO 4").

- **Gestione Dinamica dei Commi:** Poiché nei testi di legge i commi non sono sempre esplicitamente numerati, il modello è istruito a dedurli analizzando le interruzioni logiche dei paragrafi, assegnando loro una numerazione progressiva.
- **Ottimizzazione dello Schema:** Per mantenere leggeri i file JSON, il modello è programmato per non inizializzare chiavi con valori vuoti o nulli; se un livello gerarchico (come un capo o un titolo) non trova riscontro nel testo originale, la proprietà viene semplicemente omessa, fermo restando che negli atti normativi i livelli fondamentali come articoli e commi esistono necessariamente.

L'esecuzione di questo processo restituisce una directory di documenti JSON puliti e strutturati gerarchicamente, pronti per affrontare il successivo step: l'estrazione delle relazioni semantiche tra le entità legali.

4.2.2 Estrazione delle Relazioni Semantiche

La sola scomposizione del testo normativo in una struttura gerarchica ad albero rappresenta una condizione necessaria ma non sufficiente per la costruzione di un vero e proprio *Knowledge Graph*. Per sfruttare appieno la potenza topologica di un database a grafo come Neo4j, è indispensabile individuare e mappare esplicitamente i collegamenti e le dipendenze logiche che intercorrono tra le diverse disposizioni giuridiche all'interno dello stesso atto. A questo scopo, la pipeline prevede un secondo step logico interamente dedicato all'estrazione delle relazioni semantiche. In questa fase, l'input non è più il testo grezzo, ma i file JSON prodotti nello step precedente, che quindi agiscono come una base di conoscenza consolidata. Ciascun documento viene quindi analizzato dal LLM Gemini 3.5 Flash, guidato da un *System Prompt* specializzato. Tale prompt è stato progettato per riconoscere i costrutti linguistici del gergo giuridico che indicano un collegamento formale tra le parti del testo. Le relazioni estratte sono:

- **CITA:** Un articolo o comma fa riferimento a un'altra disposizione.
- **ABROGA:** Una disposizione annulla la validità giuridica di un'altra.
- **DEROGA:** Un elemento introduce un'eccezione a una regola generale.
- **SOSTITUISCE:** Una disposizione prende il posto di un'altra.
- **INTERPRETA:** La comprensione di una norma dipende dalla lettura di un'altra.

L'output di questa analisi è una rete di collegamenti formali, dove per ciascuna relazione vengono specificati il nodo di partenza, il nodo di arrivo e il tipo di legame. Queste informazioni sono esportate in un nuovo set di file JSON.

Questo approccio a due stadi permette di separare le responsabilità e ridurre il carico cognitivo del modello a ogni ciclo. In questo modo si produce un dataset pulito e affidabile, pronto per essere caricato nel grafo.

4.3 Importazione dati nei Database

Una volta completata la strutturazione dei documenti legali in formato JSON, si apre la fase di persistenza e indicizzazione dei dati. Nel contesto di questo progetto di tirocinio, l'architettura prevede il caricamento dei dati estratti all'interno del database a grafo Neo4j.

Per garantire la flessibilità e favorire un ambiente di sviluppo pulito e modulare, lo sviluppo del software ha visto l'applicazione del *Repository Pattern* e l'uso di interfacce polimorfiche.

4.3.1 I Modelli di Dominio

Prima di analizzare le strategie di persistenza, è fondamentale descrivere come i dati estratti vengano rappresentati. All'interno della cartella `models` di questo step, il sistema definisce le entità fondamentali del dominio normativo utilizzando il paradigma ad oggetti. Queste classi operano essenzialmente come *Entities*: strutture dati pure, prive di logica di persistenza diretta o dipendenze verso i DBMS, il cui unico scopo è incapsulare i dati del dominio.

L'architettura dei modelli è rappresentata nel diagramma UML della Figura 4.2. Le principali classi sono:

- **Entity**: Incapsula l'ente pubblico emittente, memorizzando i metadati di dominio.
- **LegalAct**: È la classe orchestratrice principale. Rappresenta l'atto legale nella sua interezza e contiene i metadati principali (`title`, `publication_date`, `file_name`) e soprattutto una lista `elements` che tiene traccia di tutti i sotto-elementi istanziati e delle loro relazioni gerarchiche.
- **StructuralElement**: È la classe base polimorfica per mappare le sottoclassi `Title`, `Chapter` e `Article`, mantenendo traccia del numero d'ordine e dell'eventuale rubrica.
- **Paragraph**: Rappresenta il comma ed è l'unità atomica d'informazione sulla quale verranno calcolati gli `embedding`.

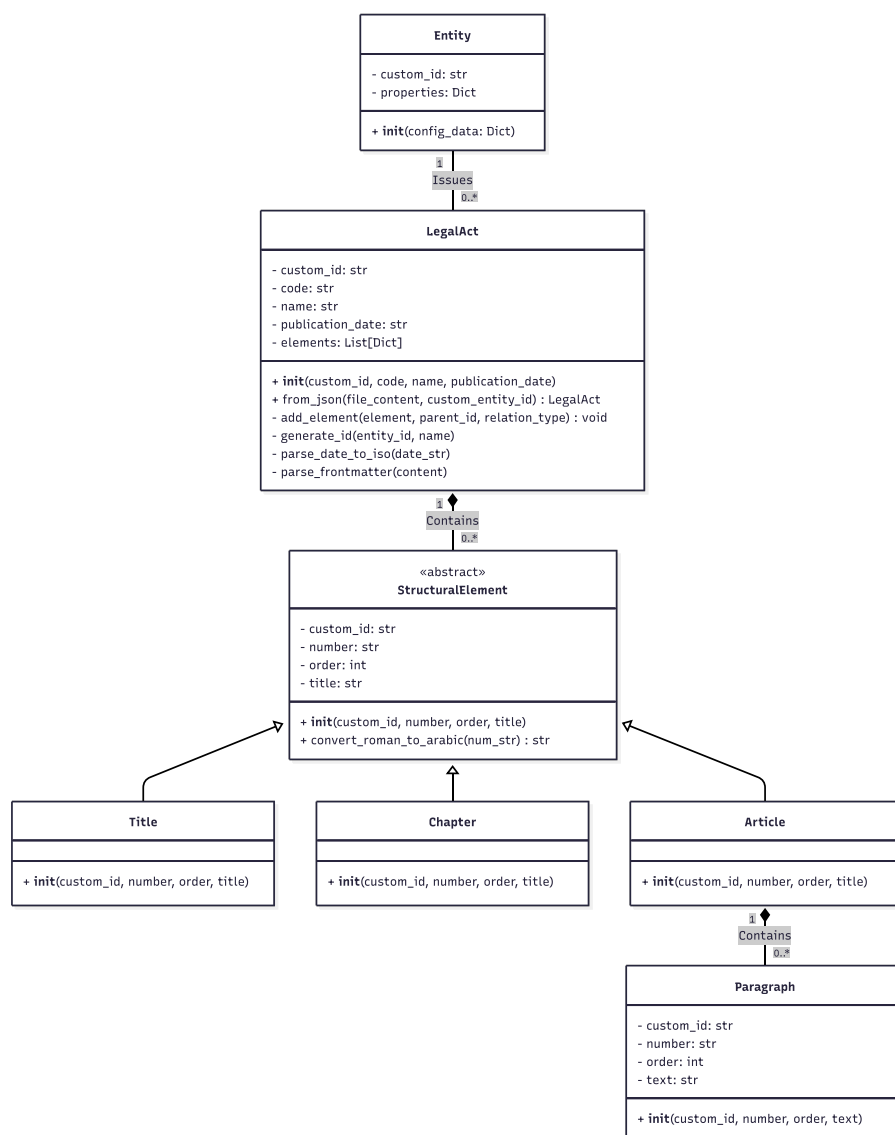


Figura 4.2: Class Diagram dei modelli di dominio

4.3.2 Interfaccia per il disaccoppiamento della persistenza

L'adozione di un'interfaccia comune di importazione nasce dalla precisa volontà di disaccoppiare la logica di lettura dei dati dalla loro effettiva persistenza fisica. Sebbene l'attuale implementazione preveda l'utilizzo esclusivo del database a grafo Neo4j, questa scelta architetturale si rivela strategica per sviluppi futuri. Qualora si rendesse necessario integrare nuovi DBMS (ad esempio, un database puramente vettoriale dedicato esclusivamente agli embedding), sarà sufficiente implementare una nuova classe concreta senza dover alterare il flusso principale del programma. Inoltre, il disaccoppiamento garantisce un'elevata testabilità del codice, permettendo di validare la logica di dominio e il parsing

dei documenti simulando le operazioni di salvataggio indipendentemente dal database sottostante.

Nel sistema sviluppato, l'interfaccia astratta di riferimento è rappresentata dalla classe `DBImporterInterface`. Essa definisce tre metodi fondamentali che ogni classe concreta deve implementare:

- `connect()`: per la gestione e la validazione della connessione con l'istanza specifica del database;
- `import_legal_act(entity, legal_act)`: per l'esecuzione effettiva di ingestion dell'atto normativo e dell'ente emittente;
- `close()`: per la chiusura sicura delle sessioni.

4.3.3 Architettura del Database Neo4j

Il database modella fedelmente l'albero gerarchico discendente del documento, garantendo la flessibilità necessaria per gestire percorsi diretti qualora i livelli intermedi siano stati omessi durante la strutturazione JSON (es. `LegalAct` → `Article` → `Paragraph`).

Le *Label* dei nodi sul grafo (ovvero **Entity**, **LegalAct**, **Title**, **Chapter**, **Article** e **Paragraph**) riflettono rigorosamente la struttura delle classi di dominio già presentate nella Sezione 4.3.1, così proprio come le *Properties* dei nodi, che conservano i metadati principali e le informazioni testuali.

Un aspetto architetturale chiave, comune a tutti i nodi, è la presenza della proprietà `custom_id`. Tale attributo codifica per intero il percorso gerarchico del nodo all'interno del documento (ad esempio, `ente__atto__title-1__chapter-2__article-3__paragraph-4`). Grazie a questo approccio, il database può sfruttare l'indicizzazione su questa proprietà per individuare immediatamente un singolo nodo, azzerando le operazioni di attraversamento del grafo e garantendo un recupero delle informazioni in tempo costante.

All'interno di questa topologia, i nodi di tipo **Paragraph** ricoprono un ruolo centrale per le funzionalità avanzate del sistema. Oltre a conservare le proprietà testuali, in questi nodi vengono salvati gli embedding semantici a dimensione variabile. L'archiviazione di tali vettori è progettata per abilitare le successive operazioni di interrogazione.

L'ecosistema di relazioni (*Edges*) all'interno del grafo si articola in tre macro-categorie progettate per supportare le interrogazioni avanzate:

- **Relazioni Strutturali:** Definiscono la topologia discendente del documento normativo. Comprendono l'arco `ISSUES` che lega l'Ente all'Atto, e la catena di contenimento gerarchico formata da `HAS_TITLE`, `HAS_CHAPTER`, `HAS_ARTICLE` e `HAS_PARAGRAPH`.
- **Relazioni Sequenziali:** I commi appartenenti allo stesso articolo sono concatenati linearmente tramite la relazione `NEXT_PARAGRAPH`. Questa architettura a lista concatenata si rivela fondamentale per espandere dinamicamente la finestra di contesto

durante la fase di interrogazione, permettendo di recuperare i commi logicamente antecedenti o successivi senza dover eseguire query computazionalmente costose per ricalcolare l'ordine strutturale.

- **Relazioni Legali:** Traducono in archi orientati sul grafo i collegamenti logici e giuridici intra-documento individuati dal prompt specializzato dell'LLM (Sezione 4.2). Le cinque tipologie di relazione estratte vengono trasposte fedelmente nelle label CITES, REPEALS, INTERPRETS, REPLACES e DEROGATES.

Occorre precisare che, sebbene lo schema dati supporti nativamente tutte le interazioni legali sopracitate, l'attuale popolamento del database non registra occorrenze per le relazioni REPEALS e REPLACES. Questa mancanza non è imputabile a un difetto del sistema di estrazione, bensì rappresenta un limite intrinseco dei documenti elaborati, che non contengono clausole esplicite di abrogazione o sostituzione strutturata.

4.3.4 Dettagli Implementativi dell'Ingestion su Neo4j

Un atto legale non può essere salvato tramite un'unica operazione atomica standard, a causa delle complesse connessioni tra i nodi, per questo motivo la responsabilità è stata distribuita su una serie di repository specializzati, ognuno deputato alla gestione del ciclo di vita di un singolo modello di dominio:

- **EntityRepository:** Gestisce l'inserimento dei nodi relativi agli enti istituzionali.
- **LegalActRepository:** Inserisce il nodo radice dell'atto e lo ancora all'ente emittente.
- **StructuralElementRepository:** Incapsula la logica Cypher necessaria a creare i nodi intermedi del grafo e gli archi verso i rispettivi nodi padri.
- **ParagraphRepository:** Si occupa esclusivamente della persistenza dei nodi foglia contenenti il testo dei commi.

Per quanto riguarda il salvataggio delle relazioni trasversali, queste informazioni vengono lette da file JSON esterni dedicati (Sezione 4.2.2), per poi essere elaborate da un metodo che, attraverso i custom_id dei nodi salvati e indicizzati sul grafo, trova in maniera quasi istantanea i nodi correlati dall'estratto JSON ed esegue una query Cypher dedicata per la creazione dell'arco semantico.

Listing 4.1 Query Cypher per l'inserimento delle relazioni legali

```
1: MATCH (s {custom_id: $source_id})
2: WITH s
3: MATCH (t {custom_id: $target_id})
4: MERGE (s)-[r:'<rel_type>']->(t)
5: RETURN r
```

L'obiettivo di questa query è rintracciare i due nodi di interesse (sorgente e destinazione) tramite il loro identificativo univoco e stabilire una relazione orientata tra di essi.

Di seguito troviamo un esempio di relazione estratta in formato JSON, con ID parzialmente omessi per motivi di leggibilità:

Listing 4.2 Esempio reale di relazione estratta in formato JSON

```

1: {
2:   "sourceId": "ente-provincia-taranto__contratto-collettivo
   -[...]__title-2__chapter-2__article-8__paragraph-6",
3:   "relationshipType": "CITES",
4:   "targetId": "ente-provincia-taranto__contratto-collettivo
   -[...]__title-2__chapter-2__article-8__paragraph-1"
5: }
```

A supporto di quanto descritto, la Figura 4.3 fornisce una rappresentazione visiva delle relazioni instaurate tra i nodi dell'articolo 8 del seguente atto: "CCNL del personale non dirigente del comparto regioni e autonomie locali"

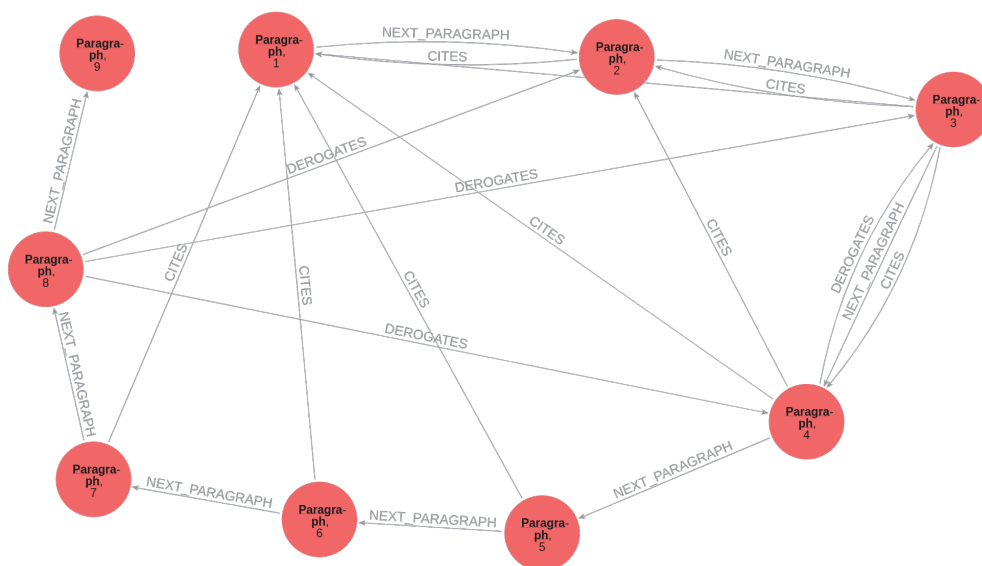


Figura 4.3: Rappresentazione visiva delle relazioni instaurate tra i nodi dell'articolo 8 di un atto normativo.

4.3.5 Risultati e Metriche

L'esecuzione della pipeline di ingestion ha dimostrato un'elevata efficienza, completando il caricamento dell'intero dataset all'interno del database in un tempo complessivo di **50.29 secondi**.

Il processo ha popolato il grafo con un totale di 8.535 nodi e 14.201 relazioni. Di seguito viene riportato il dettaglio quantitativo delle entità di dominio e degli archi salvati.

La Tabella 4.1 illustra la distribuzione dei nodi all'interno del grafo, raggruppati per *Label*. Come si può osservare dai dati, vi è una netta preponderanza numerica dei nodi Paragraph, i quali rappresentano l'unità atomica d'informazione su cui si basano le successive operazioni di embedding e ricerca.

Label Nodo	Quantità
Paragraph	6.396
Article	1.802
Chapter	147
LegalAct	95
Title	93
Entity	2

Tabella 4.1: Statistica dei nodi inseriti nel database Neo4j.

Per quanto riguarda invece le interconnessioni tra le entità, la Tabella 4.2 dettaglia il conteggio esatto degli archi strutturali, sequenziali e legali creati durante l'importazione.

Tipo di Relazione	Quantità
HAS_PARAGRAPH	6.396
NEXT_PARAGRAPH	4.594
HAS_ARTICLE	1.802
CITES	1.030
HAS_CHAPTER	147
ISSUES	95
HAS_TITLE	93
DEROGATES	41
INTERPRETS	3

Tabella 4.2: Statistica delle relazioni inserite nel database Neo4j.

Come si evince dalla Tabella 4.2, l'analisi degli archi conferma la solidità dell'architettura a lista concatenata, evidenziata dal notevole volume di relazioni NEXT_PARAGRAPH (4.594), essenziali per l'espansione dinamica del contesto in fase di retrieval.

Inoltre, analizzando sempre i medesimi dati in merito agli archi logico-legali, si osserva una netta predominanza delle relazioni di citazione (CITES, 1.030 occorrenze), seguite da un numero minoritario di deroghe (DEROGATES, 41) e interpretazioni (INTERPRETS, 3). Coerentemente con quanto anticipato nella Sezione 4.3.3, le metriche riportate confermano

l'assenza totale di occorrenze per le relazioni di abrogazione (REPEALS) e sostituzione (REPLACES) all'interno di questo specifico bacino documentale.

4.3.6 Visualizzazione del Grafo

A conclusione di questo capitolo, si propone una rappresentazione visiva di un estratto del database Neo4j volta a mostrare in maniera intuitiva la topologia della rete. Nella Figura 4.4 è possibile osservare la struttura gerarchica e le connessioni tra le diverse entità, evidenziate mediante una differenziazione cromatica:

- I **nodi gialli** rappresentano l'ente emittente (Entity);
- I **nodi azzurri** indicano gli atti legali (LegalAct);
- I **nodi rossi** corrispondono ai commi (Paragraph);
- I **restanti nodi** raffigurano le altre parti della gerarchia strutturale del documento, ossia i titoli (Title), i capi (Chapter) e gli articoli (Article).

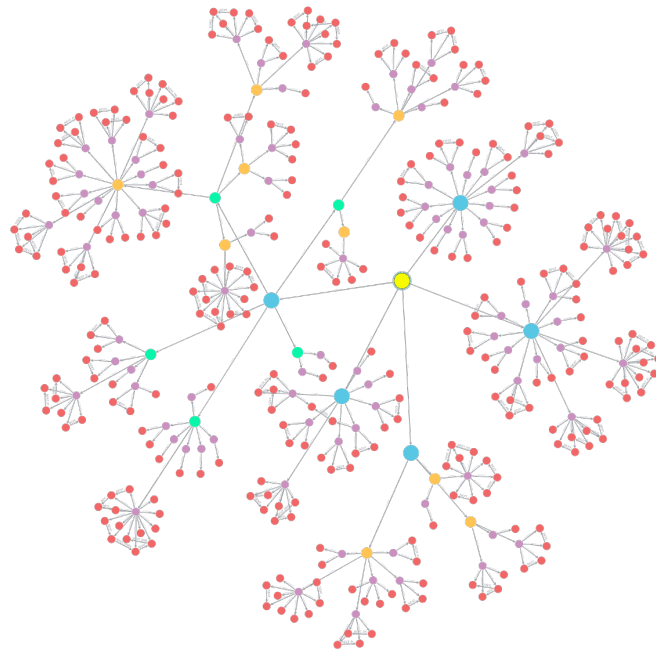


Figura 4.4: Rappresentazione visiva di un estratto del database a grafo Neo4j.

4.4 Creazione degli Embedding

L'operazione di vettorizzazione, o di *embedding*, rappresenta un passaggio cruciale per abilitare le funzionalità di ricerca semantica all'interno del database a grafo. La pipeline supporta tre modelli di embedding, dettagliati nella sezione 2.4:

- google/embeddinggemma-300m
- perplexity-ai/pplx-embed-v1-0.6b
- perplexity-ai/pplx-embed-context-v1-0.6b

Gli ultimi due appartengono alla stessa famiglia, condividono la medesima architettura di base, la dimensionalità dello spazio vettoriale e la strategia di *pooling*, ma rappresentano due configurazioni distinte. `pplx-embed-v1-0.6b` è destinato alla codifica indipendente dei testi, mentre `pplx-embed-context-v1-0.6b` è addestrato e utilizzato per produrre rappresentazioni contestualizzate di più chunk appartenenti allo stesso documento [15]. Il loro confronto nel Capitolo 6 è pertanto descrittivo: consente di osservare il comportamento delle due configurazioni, ma non costituisce un'ablazione a variabile singola. L'effetto specifico del *late chunking* richiederebbe infatti di mantenere invariati il checkpoint e la costruzione dell'input, modificando soltanto il momento in cui avviene il chunking.

I vettori vengono archiviati direttamente come proprietà all'interno dei nodi (: Paragraph) già esistenti nel grafo. Questo evita la necessità di un database vettoriale separato e consente al modello di ricerca di operare direttamente su Neo4j.

Neo4j, inoltre, supporta la creazione di indici vettoriali utilizzando algoritmi di ricerca approssimata (*Approximate Nearest Neighbor Search*, ANN). Invece di calcolare la distanza coseno tra il vettore di query e tutti gli altri vettori (operazione $O(n)$ molto costosa), questi indici riducono drasticamente lo spazio di ricerca. Di conseguenza, il tempo di esecuzione delle query passa da secondi a millisecondi, rendendo la ricerca praticabile anche su documenti normativi di grandi volumi.

4.4.1 Differenza nell'Arricchimento Contestuale: Standard vs Late Chunking

Il sistema implementa due approcci diversi per fornire *contesto* ai commi. Se si fornisce al modello solo la stringa del comma «... è punito con la reclusione da 1 a 3 anni», l'embedding generato risulterà limitato, poiché il testo manca di informazioni fondamentali come il soggetto e il reato specifico (elementi definiti nei livelli superiori come l'articolo o il titolo). Ecco come le due metodologie risolvono il problema:

A. Arricchimento Standard (Modelli: Gemma 300m e PPLX-v1)

In questo approccio, il contesto viene iniettato testualmente attraverso *String Concatenation*. Per ciascun comma, viene recuperato il contesto gerarchico risalendo l'albero

strutturale tramite query Cypher. Tutte queste informazioni vengono concatenate in un'unica stringa, producendo un testo inviato al modello simile al seguente:

```
«Atto: Codice Penale | Titolo 3 Dei delitti contro l'amministrazione  
della giustizia | Capo 2 Dei delitti contro l'autorità delle  
decisioni giudiziarie | Art. 385 Evasione | Comma 1 | ... è punito  
con la reclusione da 1 a 3 anni.»
```

In questo approccio, ogni comma viene trattato come un *chunk isolato* al momento dell'embedding. Sebbene il modello riceva il testo arricchito dai metadati gerarchici, il contesto rimane limitato a questo preambolo testuale statico.

B. Arricchimento tramite Late Chunking (Modello: PPLX-Context)

Questo approccio impiega una vera e propria contestualizzazione a livello di rete neurale. A differenza dell'approccio standard, non utilizza preamboli testuali pre-calcolati, ma affida la comprensione semantica direttamente all'algoritmo di *self-attention* del modello.

Il sistema recupera dal database, tramite query Cypher, l'intero articolo con tutti i commi che lo compongono, mantenendo traccia dell'ordine sequenziale e dei confini esatti tra di essi. Invece di frammentare il testo a monte e inviare al modello i commi singolarmente, il sistema li presenta all'encoder come un'unica sequenza di input ininterrotta.

È in questa fase che agisce il meccanismo di *late chunking*. Il modello processa l'articolo nella sua interezza attraverso i successivi layer del Transformer; in questo modo, la *self-attention* bidirezionale permette ai token di ciascun comma di "prestare attenzione" e interagire con i token adiacenti e con l'intero testo. Questo approccio abbatte i confini artificiali del chunking testuale, permettendo alla rete di risolvere dinamicamente pronomi, riferimenti incrociati e concetti impliciti basandosi sul contesto normativo globale.

Solo nella fase finale, dopo che ogni singolo token ha assorbito un significato profondamente contestualizzato, il modello applica un'operazione di *pooling* differenziata. Sfruttando gli indici sequenziali salvati in precedenza, estrae e restituisce gli embeddings separati per ciascun comma originale. Questa architettura garantisce che l'embedding finale di un singolo comma racchiuda non solo il suo significato letterale, ma anche le sfumature semantiche derivanti dal resto dell'articolo, unendo la massima precisione granulare a una perfetta consapevolezza del contesto globale.

Un esempio pratico:

Comma 1: "I soggetti che omettono la dichiarazione dei redditi sono puniti con una sanzione di 1.000 euro."

Comma 2: "La disposizione non si applica se essi dimostrano l'assenza di dolo."

Se il modello processa il Comma 2 in isolamento, la parola "essi" è ambigua e "La disposizione" è un concetto vuoto. Sfruttando il late chunking, la rete neurale collega istantaneamente "essi" del Comma 2 ai "soggetti" del Comma 1, e "La disposizione" all'intero concetto di sanzione appena letto.

–5–

Ricerca Semantica

CONTENTS: 5.1 Le due strategie e che cosa isolano. 5.2 S1: la ricerca vettoriale pura. 5.3 S2: la ricerca arricchita dalla struttura.

Il Capitolo 4 ha descritto come i documenti normativi diventano un grafo di nodi e relazioni, i cui contenuti testuali sono rappresentati mediante embedding semantici. Questo capitolo descrive che cosa accade quando una domanda in linguaggio naturale arriva a quella base di conoscenza, e definisce con precisione le due strategie di recupero che il Capitolo 6 mette a confronto sperimentalmente.

La distinzione fra le due strategie è deliberatamente stretta. Entrambe interrogano lo stesso grafo, indicizzano la stessa unità testuale, ricevono la stessa domanda codificata con lo stesso modello di embedding e restituiscono lo stesso numero di risultati. Differiscono per un solo elemento: se le relazioni giuridiche memorizzate nel grafo intervengano o meno nell'ordinamento dei risultati. Questa è la variabile indipendente dell'intero lavoro sperimentale, e l'obiettivo del presente capitolo è renderla l'unica.

5.1 Le due strategie e che cosa isolano

Il sistema espone due strategie di recupero, denominate nei log e nelle tabelle dei capitoli successivi **S1** e **S2**:

- **S1 – FullVectorSearchStrategy**: ricerca vettoriale pura. La domanda diventa un vettore, i commi vengono ordinati per similarità coseno e i primi k costituiscono il risultato. Il grafo interviene unicamente come supporto di memorizzazione: le sue relazioni non sono percorse e non influenzano l'ordinamento. Questa strategia è la *baseline*.
- **S2 – LegalGraphSearchStrategy**: ricerca ibrida. La ricerca vettoriale produce un bacino ampio di candidati; le relazioni giuridiche del grafo producono una seconda graduatoria a partire dai primi risultati vettoriali; le due graduatorie vengono

fuse. Questa è la strategia di produzione, quella che la CLI serve per impostazione predefinita.

È importante chiarire fin d'ora una proprietà che la Sezione 5.3.4 dimostrerà numericamente: **S2 non è un sistema che recupera documenti diversi da S1, è un sistema che li ordina diversamente**. Il grafo, per come è configurato, non può introdurre nel risultato finale un comma che la ricerca vettoriale non aveva già recuperato nel proprio bacino. Agisce come *re-ranker* di quel bacino. Questa non è una limitazione accidentale ma una scelta di progetto, verificata sperimentalmente nella Sezione 5.3.6.

Nessuna delle due strategie riceve informazioni sul tipo di domanda, sui documenti attesi o sulle etichette del benchmark. Il modulo di recupero riceve esclusivamente la stringa della domanda; le etichette esistono solo dentro il modulo di valutazione e non attraversano mai il confine. Questa separazione evita che il recupero utilizzi direttamente le risposte attese e rende possibile un confronto controllato fra le strategie.

5.2 S1: la ricerca vettoriale pura

La strategia di base realizza il percorso descritto in forma generale nella Sezione 4.4. La domanda viene codificata con il medesimo modello impiegato per indicizzare i commi — condizione necessaria, perché due modelli diversi producono spazi vettoriali non confrontabili — e il vettore risultante viene sottoposto a Neo4j tramite l'indice vettoriale del nodo :Paragraph.

Listing 5.1 Interrogazione dell'indice vettoriale per la ricerca di base

```
1: CALL db.index.vector.queryNodes($index_name, $top_k, $query_vector)
2: YIELD node, score
3: MATCH (act:LegalAct)-[:HAS_TITLE|HAS_CHAPTER|HAS_ARTICLE*1..4]->
4:   (art:Article)-[:HAS_PARAGRAPH]->(node)
5: RETURN node.custom_id AS paragraph_custom_id,
6:   act.title AS atto, art.number AS num_articolo,
7:   node.number AS num_comma, node.text AS testo, score
8: ORDER BY score DESC
```

L'indice restituisce i vicini approssimati del vettore di query; la risalita gerarchica successiva serve unicamente ad arricchire ciascun risultato con la sua provenienza documentale, affinché il risultato restituito all'utente sia citabile. Non modifica l'ordine.

Il costo computazionale è quello dell'*Approximate Nearest Neighbor Search*: sui 6.396 commi del corpus la latenza mediana misurata è di circa 16–20 ms per interrogazione, esclusa la codifica della domanda.

Questa strategia incarna un'ipotesi precisa sul dominio: che la pertinenza di un comma rispetto a una domanda sia catturabile dalla vicinanza semantica fra il testo della domanda e il testo del comma. È un'ipotesi ragionevole e, come mostrerà il Capitolo 6, spesso sufficiente. Il suo limite emerge quando la risposta completa richiede una disposizione

che il comma pertinente *richiama* senza riprodurne il contenuto. In quel caso il comma richiamato può essere lessicalmente e semanticamente distante dalla domanda: un encoder può ancora recuperarlo attraverso segnali presenti nel testo, ma non dispone di una rappresentazione esplicita del rinvio che ne giustifica la pertinenza.

5.3 S2: la ricerca arricchita dalla struttura

La strategia ibrida è organizzata in cinque step spiegati brevemente con lo schema seguente, e poi descritti in dettaglio nelle sottosezioni successive.

Top-200 vettoriale → relazioni giuridiche → RRF pesato → testa
riservata

5.3.1 Step 1: il bacino vettoriale

Il primo step è identico a S1, ma con un cutoff molto più ampio: vengono recuperati i **primi 200** commi per similarità coseno. Questa lista, denominata *vector_pool*, costituisce l'insieme di tutti i candidati che potranno comparire nel risultato finale.

La scelta di un bacino ampio ha una motivazione precisa. Il compito del grafo non è trovare commi che la ricerca vettoriale non vede affatto, ma *promuovere* commi che la ricerca vettoriale vede male: un comma che il coseno colloca in posizione 30 o 120, troppo in basso per essere restituito, ma che una relazione giuridica raggiunge a partire da un risultato di testa. Un bacino di 200 elementi rende visibili al meccanismo di fusione proprio quei casi.

5.3.2 Step 2: i semi del ramo giuridico

I **primi 5** commi del bacino vettoriale vengono designati come *semi (seeds)* dell'espansione a grafo. Sono gli unici nodi da cui le relazioni verranno percorse.

Il numero è piccolo, e lo è per una ragione sperimentale. L'implementazione originaria impiegava 20 semi, e il risultato era un peggioramento sistematico: una citazione che parte dal ventesimo comma migliore non è un indizio, è rumore. Il ramo giuridico ha senso solo se i punti da cui parte sono essi stessi affidabili, perché il suo compito è raggiungere ciò che è *collegato a una risposta corretta*, non ciò che è collegato a un candidato qualsiasi.

5.3.3 Step 3: il ramo giuridico

Da ciascun seme viene percorso, con un'unica interrogazione Cypher, l'insieme delle relazioni giuridiche descritte nella Sezione 4.2.2: CITES, DEROGATES, INTERPRETS, REPEALS e REPLACES. Le relazioni sono percorse in **entrambe le direzioni**: se un comma cita una disposizione, quella disposizione è pertinente al comma, ma è vero anche il contrario — una norma successiva che deroga alla norma trovata è altrettanto necessaria per rispondere correttamente.

Ogni comma raggiunto riceve un punteggio pari al coseno del seme da cui proviene, moltiplicato per un coefficiente che dipende dal tipo di relazione:

Relazione	Moltiplicatore	Natura del legame
REPEALS, REPLACES e DEROGATES	1,00	modifica l'efficacia della norma
INTERPRETS	0,90	vincola l'interpretazione
CITES	0,70	rinvio generico

Tabella 5.1: Moltiplicatori per tipo di relazione giuridica nel ramo a grafo.

La gerarchia riflette la forza giuridica del legame: una disposizione che abroga o deroga alla norma trovata è *necessaria* per una risposta corretta, mentre una citazione generica è soltanto *pertinente*.

La lista risultante viene deduplicata e troncata a $\text{top_k} \times 2$ righe.

5.3.4 Step 4: la fusione RRF pesata

Le due graduatorie — quella vettoriale e quella giuridica — vengono combinate con una variante pesata della *Reciprocal Rank Fusion*. Per ciascun comma d :

$$\text{score}(d) = \sum_{i \in \{\text{vett}, \text{leg}\}} \frac{w_i}{k_{\text{RRF}} + \text{rank}_i(d)} \quad (5.1)$$

con $k_{\text{RRF}} = 60$, $w_{\text{vett}} = 4$ e $w_{\text{leg}} = 3$. Un comma presente in una sola lista riceve un solo addendo; un comma presente in entrambe li raccoglie entrambi, e quella somma è l'intero meccanismo della strategia.

La RRF opera sui *ranghi* e non sui punteggi, il che la rende immune al fatto che un coseno e un punteggio di relazione non siano grandezze commensurabili. Inoltre, poiché l'equazione 5.1 è lineare nei pesi, soltanto il loro **rapporto** determina l'ordinamento: moltiplicare entrambi i pesi per una costante riscalda ogni punteggio e lascia l'ordine invariato. Per questa ragione i pesi sono scritti come $4/3$ e non come $0,40/0,30$: è la stessa configurazione, espressa alla precisione che le misure sostengono effettivamente.

La Tabella 5.2 mostra il comportamento concreto della formula.

Condizione del comma	Punteggio RRF	Esito
solo vettoriale, rango 1	0,006557	resta in testa
solo vettoriale, rango 10	0,005714	ultimo ammesso
solo vettoriale, rango 30	0,004444	fuori dal Top-10
vettoriale rango 30 + primo del ramo	0,009362	promosso in testa
mai recuperato, primo del ramo	0,004918	resta fuori dal Top-10

Tabella 5.2: Comportamento della fusione RRF pesata con $k_{\text{RRF}} = 60$ e pesi $4/3$.

L'ultima riga contiene la proprietà annunciata all'inizio del capitolo. Poiché $3 < 4$, un comma che il grafo raggiunge ma che la ricerca vettoriale non aveva recuperato non può mai superare un comma che essa aveva recuperato: la fusione **riordina il bacino vettoriale e non lo estende**. La soglia esatta di ingresso è un peso giuridico pari a $4 \times 61/70 = 3,49$, e restare sotto di essa è una scelta verificata (Sezione 5.3.6) e non un accidente.

5.3.5 Step 5: la testa riservata alla vettoriale

La fusione, per come è costruita, premia i commi che compaiono in entrambe le liste. Questo è l'obiettivo della strategia, ma comporta un rischio specifico: il comma che risponde *direttamente* alla domanda — quello che il coseno aveva collocato al primo posto — può essere scavalcato da un comma collegato, che il grafo ha promosso. Per una risposta completa questo è irrilevante, perché entrambi sono nel risultato; ma per un sistema a valle che legge i risultati nell'ordine in cui arrivano, non lo è affatto.

Il meccanismo che previene questo effetto è la **testa riservata**: i primi round($k \times 0,2$) ranghi del risultato finale vengono riportati in puro ordine di coseno. Sono 2 posizioni in un Top-10, 1 in un Top-5, 3 in un Top-14. Le righe sono quelle prodotte dalla fusione, con il loro punteggio e la loro provenienza; soltanto l'ordine viene ripristinato.

Una conseguenza visibile nell'output della CLI: i punteggi RRF stampati **non** sono monotonamente decrescenti lungo la testa, perché un comma promosso subito al di sotto di essa può portare un punteggio superiore a quello del primo risultato. Non è un errore di ordinamento, è la riserva che agisce.

L'efficacia di questo componente è netta e viene riportata per intero nel Capitolo 6. Va però anticipato che si tratta di uno scambio dichiarato e non di un guadagno gratuito: le righe che la fusione promuoverebbe nei primi due ranghi sono commi attesi nel **49,1%** dei casi, contro il **31,2%** delle righe che la testa mantiene. La testa cede quindi un po' di copertura in cima per riportare al primo posto il comma che effettivamente risponde.

5.3.6 Perché il peso giuridico resta 3

Il rapporto fra i due pesi della fusione è stato esplorato sistematicamente su diciannove valori, dal rapporto 0,375 al rapporto 1,5, sulle domande con risposta e su tutti e tre i modelli di embedding.

Il risultato più istruttivo è un'assenza di risultato: fra i rapporti da 0,625 a 1,5 — quindici punti su diciannove — lo split di selezione restituisce la *medesima* graduatoria. In ogni punto, nessun risultato incompleto diventa completo e nessun risultato completo diventa incompleto. Lo split interrogato sul rapporto non ha opinione, ed è precisamente questa la ragione per cui il valore adottato è scritto con una cifra e non con tre.

La domanda vera dello *sweep* era però un'altra: che cosa accade superando la soglia di ingresso di 3,49, cioè permettendo al grafo di aggiungere candidati anziché limitarsi a riordinare. La risposta è misurata: al di sopra della soglia entrano effettivamente 21 righe di sola provenienza giuridica su 3.450 restituite, e **2 di esse sono commi attesi**:

una precisione del 9,5% contro il 13,7% delle righe che spodestano. Consentire al grafo di ampliare il bacino peggiora la riga media, a fronte di un guadagno complessivo di tre decimi di punto. Stare sotto la soglia è il lato giusto su cui trovarsi.

Un'ultima osservazione, che il capitolo successivo riprenderà: la metrica che misura il rango della prima risposta corretta assume lo stesso valore in **tutti e diciannove** i punti dello *sweep*, fino all'ultimo decimale. Quel rango appartiene interamente alla testa riservata, e nessun peso della fusione lo tocca.

5.3.7 Un ramo rimosso: l'adiacenza fra commi

Accanto al ramo giuridico ha operato per un certo periodo un secondo ramo, che percorreva la relazione NEXT_PARAGRAPH per raggiungere i commi contigui a quelli trovati. L'intuizione è forte: se la risposta sta nel comma 3 di un articolo, è probabile che il comma 4 la completi.

È stata misurata a tutti i pesi che il ramo aveva storicamente assunto, e il risultato andava sempre a peggiorare: peso 0,20 peggiore di 0,10, peso 0,10 peggiore di zero. Rimuoverlo vale **+0,6 punti** di completezza a Top-10 e **+2,6** a Top-5. Misurati isolatamente, il ramo giuridico da solo raggiunge l'83,8% di completezza a Top-10, mentre l'adiacenza da sola raggiunge il 71,9%, cioè *al di sotto* del 72,2% della sola graduatoria vettoriale.

Nel complesso, le scelte descritte in questo capitolo definiscono le due strategie di recupero e isolano il contributo delle relazioni giuridiche al riordinamento dei risultati. Il Capitolo 6 presenta i risultati dell'esperimento e mette a confronto quantitativamente le prestazioni delle due strategie.

–6–

Valutazione Sperimentale

CONTENTS: 6.1 Il banco di domande. 6.2 Le metriche. 6.3 Risultati sul benchmark diagnostico. 6.4 Il costo della struttura. 6.5 Il contributo del grafo con un encoder contestuale. 6.6 Limiti della valutazione. 6.7 Sintesi del capitolo.

Questo capitolo misura le due strategie di recupero definite nel Capitolo 5 e risponde alla domanda che motiva l'intero lavoro: *la struttura esplicita di un grafo normativo migliora il recupero rispetto alla sola similarità semantica, e a quale prezzo?*

La Sezione 6.5 verifica infine se il contributo del grafo persista quando la ricerca vettoriale usa rappresentazioni già contestualizzate a livello di articolo.

6.1 Il banco di domande

Non esiste, per il corpus normativo impiegato in questo lavoro, un insieme di domande e risposte di riferimento. È stato quindi necessario costruirne uno, e la sua composizione è una scelta metodologica che condiziona ogni numero riportato nel seguito.

6.1.1 Struttura del banco

Il benchmark diagnostico è costituito da **70 domande**, organizzate in **sette categorie da dieci**. Ogni domanda dichiara l'insieme dei commi che costituiscono la risposta completa, identificati dal loro `custom_id` nel grafo. Le etichette sono usate esclusivamente per la valutazione e non raggiungono mai il modulo di recupero. La Tabella 6.1 riassume la composizione del banco e il meccanismo che ciascuna categoria mette alla prova.

Categoria	n	Che cosa mette alla prova
SEMANTIC_DIRECT	10	la domanda è vicina al testo
SEMANTIC_INDIRECT	10	la domanda esprime indirettamente il contenuto del testo
RELATIONAL_CITES_LOW_SIMILARITY	10	rinvii i cui estremi hanno testi poco simili
RELATIONAL_CITES_HIGH_SIMILARITY	10	rinvii i cui estremi hanno testi molto simili, come condizione di non-effetto atteso
RELATIONAL_DEROGATES	10	una regola generale e la disposizione che vi deroga
MULTI_ARTICLE	10	commi attesi distribuiti su più articoli
MULTI_PARAGRAPH	10	più commi dello stesso articolo

Tabella 6.1: Composizione del benchmark diagnostico.

Le due categorie sulla relazione CITES

Le due categorie dedicate ai rinvii condividono la stessa struttura: il comma sorgente contiene un rinvio esplicito a una disposizione collocata in un altro articolo dello stesso atto. I casi sono suddivisi in due gruppi in base alla similarità semantica tra il comma sorgente e quello richiamato.

La categoria RELATIONAL_CITES_LOW_SIMILARITY comprende i casi in cui i due commi risultano semanticamente distanti. In questa condizione, una ricerca basata esclusivamente sulla similarità semantica può avere difficoltà a recuperarli congiuntamente, poiché il rinvio collega disposizioni che svolgono funzioni differenti o trattano aspetti diversi della stessa disciplina. La presenza di un arco esplicito nel grafo consente invece, una volta individuato il comma sorgente, di raggiungere direttamente la disposizione richiamata, anche quando la somiglianza tra i rispettivi contenuti testuali è ridotta.

Un esempio è rappresentato dal rinvio relativo all'anticipazione delle spese di missione, contenuto nel *Regolamento di contabilità della Provincia di Parma*. L'articolo 74, comma 2, dispone:

“L'anticipazione viene corrisposta all'avente diritto secondo le modalità previste al comma 3 dell'art. 73, previa presentazione della copia del relativo atto autorizzativo allo svolgimento della trasferta o missione.”

La disposizione richiamata, contenuta nell'articolo 73, comma 3, stabilisce invece:

“La spesa viene disposta su specifica richiesta firmata dal Dirigente competente tramite l'emissione di appositi buoni economici in cui deve comparire obbli-

gatoriamente l'oggetto della spesa, il suo ammontare, il soggetto percepente la somma, l'impegno di spesa al quale è riferita la spesa."

Il primo comma riguarda principalmente l'anticipazione connessa a una missione, mentre il secondo definisce una procedura amministrativo-contabile più generale. La relazione deriva quindi soprattutto dall'importazione, mediante rinvio, di una procedura definita altrove, non da una forte somiglianza tra i due testi.

La categoria RELATIONAL_CITES_HIGH_SIMILARITY comprende invece casi nei quali il rinvio collega disposizioni semanticamente e lessicalmente più vicine. In questa condizione ci si attende un contributo minore dalla struttura a grafo, poiché la disposizione richiamata presenta già caratteristiche che ne favoriscono il recupero attraverso la ricerca vettoriale.

Un esempio riguarda l'utilizzo del mezzo proprio da parte degli amministratori ed è tratto dal *Regolamento per la disciplina dei rimborsi delle spese di viaggio e delle missioni istituzionali degli amministratori della Provincia di Parma*. L'articolo 7, comma 2, afferma:

"Nel caso di utilizzo del proprio mezzo, la Provincia di Parma non concorrerà alla copertura assicurativa di detto veicolo e rimarrà comunque esente da ogni responsabilità civile, penale e per infortuni, in relazione ai danni che l'uso del mezzo stesso potrà arrecare a cose o persone, in particolare a terzi, all'Amministratore che utilizza il proprio mezzo, ai trasportati o al mezzo medesimo, come già disciplinato all'art. 5, comma 4, del presente Regolamento."

La disposizione richiamata, contenuta nell'articolo 5, comma 4, stabilisce:

"Gli Amministratori, autorizzati all'uso del mezzo proprio, devono sollevare l'Amministrazione da qualsiasi responsabilità dichiarando, nell'apposito predisposto modello: [...] che il veicolo utilizzato è in regola con le norme previste per l'assicurazione dei veicoli; [...] di essere in possesso di regolare e valida patente di guida e/o abilitazione alla guida dei veicoli ai sensi del vigente Codice della strada."

I due commi condividono il soggetto, la situazione disciplinata e concetti centrali quali l'utilizzo del mezzo proprio, la regolarità del veicolo e l'esclusione della responsabilità dell'Amministrazione. Il rinvio collega pertanto disposizioni caratterizzate da un'elevata vicinanza sia tematica sia lessicale.

Il trattamento delle altre relazioni giuridiche

La stessa suddivisione non è stata applicata a DEROGATES. Nei casi presenti nel corpus, questa relazione collega prevalentemente disposizioni che disciplinano lo stesso istituto secondo una struttura regola-eccezione. La disposizione derogante e quella derogata risultano quindi generalmente vicine sul piano tematico e spesso anche su quello lessicale, pur svolgendo funzioni normative differenti. Un esempio riguarda le auto di servizio disciplinate dal *Regolamento per la gestione del parco veicolare della Provincia di Parma*. La disciplina generale, contenuta nell'articolo 6, commi 7 e 8, prevede:

“Le auto di servizio non possono essere utilizzate per il trasporto dall’abitazione all’ufficio o al luogo di lavoro, fatto salvo il personale che svolge servizio di reperibilità.”

“Le stesse dovranno di norma essere stazionate nel luogo di assegnazione corrispondente alla sede dell’Ufficio o del Centro Operativo cui appartiene il dipendente o di quello più vicino alla propria residenza.”

La disposizione derogante, contenuta nell’articolo 7, comma 1, stabilisce:

“Per i dipendenti in servizio di reperibilità è consentito l’utilizzo del mezzo per motivi di servizio in deroga alle disposizioni che prevedono il ricovero presso le sedi della Provincia.”

Le disposizioni condividono i concetti centrali: le auto di servizio, il personale in reperibilità, l’utilizzo del mezzo e il ricovero presso le sedi della Provincia. La loro affinità semantica non implica tuttavia equivalenza: i commi dell’articolo 6 definiscono la disciplina generale, mentre il comma dell’articolo 7 ne limita l’applicazione introducendo un’eccezione.

Questo esempio evidenzia una differenza rispetto a CITES. Una citazione può collegare sia disposizioni fortemente affini sia disposizioni semanticamente distanti. Nei casi DEROGATES osservati nel corpus, invece, la struttura regola–eccezione tende a mantenere una forte condivisione dell’oggetto normativo. Una suddivisione in condizioni di alta e bassa similarità avrebbe pertanto un interesse sperimentale più limitato e sarebbe ulteriormente indebolita dal numero ridotto di archi DEROGATES disponibili.

Infine, non sono state definite slice analoghe per INTERPRETS, REPEALS e REPLACES poiché, nel grafo utilizzato per la sperimentazione, sono presenti soltanto tre archi INTERPRETS, mentre non risultano archi REPEALS o REPLACES. Tali quantità non consentono di costruire campioni sufficientemente ampi, né di suddividerli ulteriormente in condizioni di alta e bassa similarità. L’introduzione di slice dedicate non avrebbe pertanto prodotto condizioni sperimentali sufficientemente solide per il benchmark.

6.1.2 Etichette primarie e di supporto

Ogni domanda distingue due categorie di commi attesi:

- i commi **primari**, cioè quelli che rispondono direttamente alla domanda e che una ricerca semantica può plausibilmente individuare per vicinanza testuale;
- i commi **di supporto**, cioè le disposizioni necessarie a completare la risposta e la cui pertinenza deriva da un collegamento normativo o strutturale con i commi primari.

Questa separazione è lo strumento diagnostico centrale del capitolo, perché consente di osservare separatamente il recupero della risposta diretta e quello del contesto che la

completa. La ricerca vettoriale può recuperare entrambe le categorie; l'ipotesi sottoposta a verifica è che l'impiego esplicito delle relazioni produca un guadagno soprattutto sulla copertura dei commi *di supporto*.

6.2 Le metriche

Tutte le metriche sono calcolate sul Top-K restituito, ai cutoff 5 e 10. Ciascun cutoff è una selezione a sé: sia i bacini del ramo sia la testa riservata scalano con k , quindi un Top-5 non è necessariamente il prefisso di un Top-10

- **Complete HIT@K**: la domanda conta come risolta soltanto se *tutti* i commi attesi si trovano entro il cutoff. È la metrica più severa e la più aderente al caso d'uso, perché una risposta normativa parziale può essere peggiore di nessuna risposta: un utente che riceve la regola generale senza la deroga che la disattiva riceve un'informazione scorretta.
- **Copertura gold@K**: la frazione dei commi attesi presenti nel cutoff. A differenza della precedente premia i progressi parziali.
- **Copertura supporting@K**: la stessa quantità, ristretta ai soli commi di supporto. Il confronto fra S1 e S2 su questa metrica indica quanto l'impiego delle relazioni aiuti a recuperare il contesto che completa la risposta.
- **MRR@K**: il reciproco del rango della prima risposta corretta. Misura la qualità dell'*ordinamento*, non della selezione, ed è la metrica che un sistema generativo a valle consuma effettivamente, perché legge i risultati nell'ordine in cui arrivano.

Tutte le medie sono calcolate prima dentro ciascun modello e poi fra i modelli, così che i tre modelli pesino ugualmente. Dove due strategie vengono confrontate, il confronto è **appaiaito**: per ogni valutazione (domanda $\times$ modello) si osserva se il passaggio da S1 a S2 trasformi un risultato incompleto in completo, un risultato completo in incompleto oppure lasci invariato l'esito.

6.2.1 Configurazione sperimentale

Le misure riportate provengono dall'esecuzione 20260918T180810.030837Z sul benchmark diagnostico.

I modelli di embedding confrontati sono i tre già introdotti nella Sezione 4.4, e il capitolo li impiega in **due ruoli distinti** che vanno dichiarati prima di leggere qualunque tabella che li elenca fianco a fianco, perché altrimenti la stessa riga sembra dire due cose diverse in due punti diversi del capitolo.

Primo ruolo, nelle Sezioni 6.3 e 6.4: tre verifiche di robustezza. La domanda posta lì è se la struttura migliori il recupero, ed è ripetuta con ciascun encoder per verificare che il segno del risultato non dipenda da una sola scelta del modello.

Secondo ruolo, nella Sezione 6.5: verifica con encoder contestuale fissato. Il test mantiene fisso `pplx-embed-context-v1-0.6b` e misura se il passaggio da S1 a S2 produca ancora un guadagno. Il confronto con `pplx-embed-v1-0.6b` resta soltanto descrittivo: i due modelli differiscono per addestramento e modalità di costruzione dell'input, quindi non costituiscono un'ablazione a variabile singola del *late chunking*.

6.3 Risultati sul benchmark diagnostico

Metrica	S1	S2	delta
Complete HIT@10	62,4%	79,5%	+17,1 pt
Complete HIT@5	47,1%	66,2%	+19,0 pt
Copertura gold@10	80,4%	88,0%	+7,6 pt
Copertura gold@5	70,0%	79,1%	+9,1 pt
Copertura supporting@10	69,8%	82,0%	+12,2 pt
MRR@10	79,8%	79,5%	-0,4 pt

Tabella 6.2: Confronto complessivo fra ricerca vettoriale pura (S1) e ricerca arricchita dalla struttura (S2), su 70 domande e tre modelli di embedding.

Sulla composizione scelta per il benchmark, la strategia ibrida ottiene valori superiori su ogni metrica di selezione. Il dato aggregato non va però interpretato separatamente dalle categorie: combina condizioni nelle quali le relazioni possono contribuire e condizioni nelle quali il meccanismo non è richiesto, con pesi stabiliti dalla costruzione del banco. L'unica metrica in cui S2 non migliora è l'MRR, che diminuisce di quattro decimi di punto a causa del costo di ordinamento introdotto dalla fusione.

Sulle 210 valutazioni ottenute combinando 70 domande e tre encoder, il passaggio da S1 a S2 trasforma **39 risultati incompleti in completi** e **3 risultati completi in incompleti**; nelle restanti 168 valutazioni l'esito non cambia. Il conteggio rende ispezionabili i casi di disaccordo, ma va letto come statistica descrittiva: le tre valutazioni associate alla stessa domanda sono correlate e non costituiscono osservazioni indipendenti.

6.3.1 Il guadagno dipende dalla categoria e non segue interamente l'attesa iniziale

Categoria	S1	S2	delta
RELATIONAL_CITES_LOW_SIMILARITY	53,3%	90,0%	+36,7 pt
RELATIONAL_CITES_HIGH_SIMILARITY	60,0%	90,0%	+30,0 pt
RELATIONAL_DEROGATES	70,0%	86,7%	+16,7 pt
MULTI_ARTICLE	30,0%	70,0%	+40,0 pt
MULTI_PARAGRAPH	60,0%	66,7%	+6,7 pt
SEMANTIC_INDIRECT	73,3%	73,3%	0,0 pt
SEMANTIC_DIRECT	90,0%	80,0%	-10,0 pt

Tabella 6.3: Complete HIT@10 per categoria.

La Tabella 6.3 è più informativa della media che la riassume. Il guadagno non è diffuso: è concentrato sulle categorie costruite attorno a un rinvio o a una distribuzione della risposta su più articoli, cioè dove il meccanismo può agire. Questa regolarità non coincide però interamente con la previsione iniziale: anche RELATIONAL_CITES_HIGH_SIMILARITY, costruita come condizione di non-effetto atteso, guadagna 30 punti di Complete HIT@10. Sulle domande semantiche indirette il grafo è invece **esattamente neutro**, e sulle domande semantiche dirette **costa dieci punti**.

La scomposizione del controllo chiarisce il risultato. La copertura dei commi di supporto passa dal 63,3% al 93,3%. In nove valutazioni il passaggio da S1 a S2 trasforma un risultato incompleto in completo: S1 aveva già collocato il comma primario nella Top-10, ma non la disposizione richiamata necessaria a completare la risposta; S2 promuove quest'ultima ai ranghi 3-7. Il miglioramento osservato riguarda quindi, in questi casi, il recupero del contesto normativo collegato necessario a completare la risposta. Non si osserva il cambiamento inverso nella categoria.

La previsione di non-effetto non è quindi confermata rispetto alla completezza della risposta. L'alta similarità fra i due estremi dell'arco non garantisce che entrambi compaiano nella Top-10 quando competono con l'intero corpus. Il controllo mette così in evidenza una funzione più circoscritta del grafo: non necessariamente individuare il primo comma pertinente, ma completare il contesto normativo a partire da esso. Questo risultato resta diagnostico, perché la categoria è costruita attorno a rinvii che il sistema può percorrere e non ne misura la frequenza nel traffico reale.

Quest'ultima riga non è un difetto della misura, è la misura funzionante. Le domande dirette sono quelle in cui la ricerca vettoriale già risponde correttamente nel 90% dei casi; su di esse il grafo non ha nulla da aggiungere, e gli slot che la fusione riserva a candidati di provenienza giuridica sostituiscono risultati vettoriali corretti. È il costo di eseguire un meccanismo dove non serve: la misura complessiva non può essere interpretata come un

guadagno netto, ma come un guadagno in condizioni favorevoli e un costo in condizioni sfavorevoli.

6.3.2 Il segno del risultato si ripete sui tre encoder

Modello di embedding	S1	S2	delta
embeddinggemma-300m	54,3%	77,1%	+22,9 pt
pplx-embed-v1-0.6b	65,7%	82,9%	+17,1 pt
pplx-embed-context-v1-0.6b	67,1%	78,6%	+11,4 pt

Tabella 6.4: Complete HIT@10 per modello di embedding, sul banco principale.

Ciò che è invariante è il **segno** del delta: tutti e tre gli encoder migliorano con la struttura, e nessuno peggiora. Ciò che non è invariante è l'**ampiezza**: il modello che parte dal valore più basso riceve il guadagno maggiore, mentre quello contestuale riceve il guadagno minore (+11,4 punti). La tabella mostra quindi che, nelle condizioni del benchmark, nessuno dei tre encoder rende superfluo il contributo della struttura; non dimostra invece che tale contributo sia indipendente dall'encoder in senso forte.

6.4 Il costo della struttura

Un risultato che riporta soltanto i guadagni non è una misura. Nella configurazione valutata il grafo presenta due costi osservabili: uno sull'ordinamento e uno sul tempo di ricerca.

6.4.1 Il costo di ordinamento

Il primo costo è l'MRR: -0,4 punti. È piccolo, ma la sua origine è istruttiva. La fusione promuove i commi che compaiono in entrambe le liste, e questi sono tipicamente commi *collegati* alla risposta piuttosto che la risposta stessa; il comma che risponde direttamente può quindi scivolare dal primo posto. La testa riservata esiste per contenere questo effetto, e la sua efficacia è misurabile disattivandola: sul benchmark il rango medio della prima risposta corretta perde oltre cinque punti di MRR, a copertura invariata. È quindi il principale presidio della pipeline contro il deterioramento dell'ordinamento.

6.4.2 Il costo computazionale

Il secondo costo è il tempo. La strategia ibrida aggiunge un percorso di andata e ritorno verso il grafo e la fusione di due graduatorie.

Strategia	cutoff	mediana	media	p95
S1	Top-10	16 ms	16 ms	20 ms
S2	Top-10	18 ms	21 ms	35 ms

Tabella 6.5: Latenza di `strategy.search()`, mediata sui tre modelli. Esclude la codifica della domanda.

Il sovracosto è di circa 2–5 ms sulla mediana e di 15 ms sul novantacinquesimo percentile. In termini relativi è un incremento sensibile; in condizioni hardware e software della misura, il suo valore assoluto rimane contenuto rispetto alla codifica della domanda e alla successiva generazione della risposta. La valutazione consente quindi di considerare sostenibile il costo computazionale della struttura nella configurazione sperimentata, senza generalizzare la misura ad altri ambienti.

6.5 Il contributo del grafo con un encoder contestuale

Il grafo e il *late chunking* possono entrambi compensare la perdita di contesto prodotta dalla suddivisione di un documento in unità atomiche, ma impiegano segnali differenti. Il *late chunking* incorpora nella rappresentazione di ciascun comma informazioni provenienti dal resto dell'articolo; il grafo codifica invece relazioni esplicite fra disposizioni e le usa durante la ricerca. La parziale sovrapposizione solleva una verifica di robustezza rilevante per il risultato principale: *se la rappresentazione vettoriale è già contestualizzata, le relazioni esplicite conservano un contributo osservabile?*

Metrica con encoder contestuale	S1	S2	delta
Complete HIT@10	67,1%	78,6%	+11,4 pt
Copertura gold@10	81,3%	86,8%	+5,5 pt
Copertura supporting@10	71,9%	79,7%	+7,8 pt
MRR@10	75,2%	74,8%	−0,5 pt

Tabella 6.6: Contributo della struttura mantenendo fisso l'encoder contestuale.

Sull'intero banco di 70 domande, la struttura guadagna 11,4 punti di Complete HIT@10 anche quando i commi sono rappresentati mediante l'encoder contestuale. La copertura dei commi gold aumenta di 5,5 punti e, sulle domande per le quali sono definiti commi di supporto, la loro copertura cresce di 7,8 punti. L'MRR diminuisce invece di 0,5 punti, riproducendo su scala ridotta il costo di ordinamento osservato nel confronto aggregato. Il risultato respinge quindi, nelle condizioni valutate, l'ipotesi forte secondo cui la contestualizzazione dell'encoder assorbirebbe interamente il contributo delle relazioni esplicite.

Il risultato aggregato non chiarisce tuttavia in quali condizioni il grafo mantenga il proprio vantaggio. La scomposizione per categoria mostra che l'effetto non consiste in un miglioramento uniforme.

Categoria	S1	S2	delta
RELATIONAL_CITES_LOW_SIMILARITY	50,0%	80,0%	+30,0 pt
RELATIONAL_CITES_HIGH_SIMILARITY	50,0%	80,0%	+30,0 pt
RELATIONAL_DEROGATES	90,0%	100,0%	+10,0 pt
MULTI_ARTICLE	50,0%	70,0%	+20,0 pt
MULTI_PARAGRAPH	70,0%	70,0%	0,0 pt
SEMANTIC_INDIRECT	80,0%	80,0%	0,0 pt
SEMANTIC_DIRECT	80,0%	70,0%	-10,0 pt

Tabella 6.7: Complete HIT@10 per categoria mantenendo fisso l'encoder contestuale.

Il vantaggio di S2 si concentra nelle categorie relazionali e in MULTI_ARTICLE; MULTI_PARAGRAPH e SEMANTIC_INDIRECT rimangono invariati, mentre SEMANTIC_DIRECT perde dieci punti. Questa distribuzione è coerente con quanto osservato nel confronto per categoria (Tabella 6.3), tranne che per MULTI_PARAGRAPH: nel confronto precedente beneficia della struttura, ma con l'encoder contestuale non mostra alcun guadagno aggiuntivo.

Fra le condizioni neutrali, MULTI_PARAGRAPH costituisce il controllo più direttamente pertinente al rapporto fra grafo e *late chunking*: richiede di recuperare più commi dello stesso articolo, ossia informazioni comprese nella medesima finestra di contestualizzazione dell'encoder. Se il grafo compensasse principalmente la perdita di contesto intra-articolo, il suo contributo dovrebbe quindi ridursi, o scomparire, in questa categoria.

Metrica su MULTI_PARAGRAPH	S1	S2	delta
Complete HIT@10	70,0%	70,0%	0,0 pt
Copertura gold@10	76,7%	76,7%	0,0 pt
Copertura supporting@10	75,0%	75,0%	0,0 pt
MRR@10	65,8%	65,8%	0,0 pt

Tabella 6.8: Confronto fra S1 e S2 sulle dieci domande MULTI_PARAGRAPH, mantenendo fisso l'encoder contestuale.

S1 e S2 ottengono gli stessi risultati su tutte le metriche considerate. In questa categoria non emerge quindi un vantaggio ulteriore del grafo. Il pareggio è coerente con una sovrapposizione dei due segnali quando il contesto necessario è interno allo stesso articolo; la Tabella 6.7 mostra però che il grafo conserva un contributo nelle altre categorie, ossia quelle che coinvolgono relazioni esplicite o più articoli.

Le due tecniche operano su forme di contesto differenti. Il *late chunking* contestualizza ciascun comma mediante le informazioni contenute nella finestra ricevuta dall'encoder, che in questa implementazione coincide con l'articolo. La ricerca vettoriale continua a interrogare l'intero corpus, ma ogni rappresentazione incorpora direttamente soltanto il contesto intra-articolo.

Il grafo codifica invece relazioni esplicite tra disposizioni, indipendentemente dal fatto che appartengano o meno allo stesso articolo. Durante la ricerca, tali relazioni possono essere impiegate per promuovere candidati presenti nel bacino vettoriale. Il segnale strutturale può quindi estendersi oltre il perimetro contestualizzato dall'encoder.

I risultati sono coerenti con questa distinzione. In MULTI_PARAGRAPH, dove i commi attesi appartengono allo stesso articolo, l'aggiunta del grafo non produce un beneficio osservabile. Il vantaggio rimane invece nelle categorie relazionali e in MULTI_ARTICLE, cioè in condizioni nelle quali la risposta può coinvolgere collegamenti o contenuti distribuiti oltre il singolo articolo. Nel benchmark considerato emerge quindi una parziale sovrapposizione sul contesto intra-articolo, ma non una completa ridondanza tra contestualizzazione e relazioni esplicite.

6.6 Limiti della valutazione

Prima di sintetizzare i risultati complessivi, è opportuno chiarire ciò che le misure adottate non consentono di stabilire.

La dimensione del campione è modesta. Settanta domande consentono di descrivere differenze ampie, ma non di attribuire un significato forte a scarti di pochi punti. Le 210 combinazioni domanda $\times$ modello non sono osservazioni indipendenti, perché i tre encoder vengono valutati sugli stessi 70 quesiti.

Il benchmark è costruito, non campionato. Cinque categorie su sette sono state scritte attorno a un meccanismo del sistema. I risultati mostrano quindi che cosa accade nelle condizioni rappresentate dalle categorie, ma non stimano la frequenza di tali condizioni nel traffico reale.

Il corpus è uno solo. Le proporzioni fra i tipi di relazione di questo grafo — 1.030 archi CITES contro 41 DEROGATES e 3 INTERPRETS, con REPEALS e REPLACES assenti — determinano quanto il ramo giuridico possa contribuire. Un corpus con una fitta trama di abrogazioni e sostituzioni si comporterebbe diversamente, e presumibilmente in favore della struttura, ma questo non è stato misurato.

La qualità della risposta generata non è misurata. Tutto ciò che questo capitolo misura è il recupero: quali commi arrivano, in quale ordine. Che un sistema generativo a valle produca con quei commi una risposta corretta è una proprietà distinta, che richiederebbe un impianto di valutazione diverso.

6.7 Sintesi del capitolo

Considerando tutte le 70 domande, la ricerca arricchita dal grafo porta la Complete HIT@10 dal 62,4% al 79,5%, con un guadagno di 17,1 punti. Sulle 210 valutazioni ottenute combinando le domande con i tre encoder, il passaggio da S1 a S2 trasforma 39 risultati incompleti in completi e 3 risultati completi in incompleti; nelle restanti 168 valutazioni l'esito non cambia. Il vantaggio riguarda la completezza della selezione, mentre l'MRR diminuisce di 0,4 punti: il grafo recupera più spesso l'intera risposta, ma può modificare l'ordine dei primi risultati.

La verifica con encoder contestuale fissato conduce alla stessa conclusione generale: S2 supera S1 di 11,4 punti di Complete HIT@10, di 5,5 punti nella copertura dei commi gold e di 7,8 punti nella copertura dei commi di supporto, a fronte di una diminuzione di 0,5 punti di MRR. Il vantaggio si concentra nelle categorie relazionali e in MULTI_ARTICLE; MULTI_PARAGRAPH e SEMANTIC_INDIRECT rimangono invariati, mentre SEMANTIC_DIRECT peggiora. Il *late chunking* e il grafo risultano quindi parzialmente sovrapposti sul contesto intra-articolo, ma non ridondanti rispetto all'insieme delle relazioni rappresentate nel benchmark.

Nel complesso, l'esperimento mostra che la struttura può aumentare la completezza del recupero nelle condizioni per cui le relazioni risultano pertinenti, con un costo contenuto ma osservabile sull'ordinamento. Le conclusioni descrivono il comportamento dei meccanismi nelle condizioni rappresentate dal benchmark e non costituiscono una stima delle prestazioni attese su domande reali.

Bibliografia

- [1] *Neo4j Graph Platform*. URL: <https://neo4j.com>.
- [2] Nicholas Pipitone e Ghita Hour Alami. *LegalBench-RAG: A Benchmark for Retrieval-Augmented Generation in the Legal Domain*. 2024. arXiv: 2408.10343 [cs.IR]. URL: <https://arxiv.org/abs/2408.10343>.
- [3] Aidan Hogan et al. «Knowledge Graphs». In: *ACM Computing Surveys* 54.4 (2021), pp. 1–37. DOI: 10.1145/3447772.
- [4] Fabio Vitali, Monica Palmirani, Roger Sperberg e Véronique Parisse. *Akoma Ntoso Version 1.0. Part 2: Specifications*. OASIS Standard. 2018. URL: <https://docs.oasis-open.org/legaldocml/akn-core/v1.0/os/part2-specs/akn-core-v1.0-os-part2-specs.html>.
- [5] Patrick Lewis et al. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. 2020. URL: <https://arxiv.org/abs/2005.11401>.
- [6] Darren Edge et al. *From Local to Global: A Graph RAG Approach to Query-Focused Summarization*. 2024. DOI: 10.48550/arXiv.2404.16130. arXiv: 2404.16130 [cs.CL]. URL: <https://arxiv.org/abs/2404.16130>.
- [7] *Open Data Loader*. URL: <https://github.com/opendataloader-project/opendataloader-pdf>.
- [8] Christoph Auer et al. *Docling Technical Report*. 2024. DOI: 10.48550/arXiv.2408.09869. arXiv: 2408.09869 [cs.CL]. URL: <https://arxiv.org/abs/2408.09869>.
- [9] *RapidOCR: OCR Engine*. URL: <https://github.com/RapidAI/RapidOCR>.
- [10] Neo4j, Inc. *Vector Indexes*. 2026. URL: <https://neo4j.com/docs/cypher-manual/current/indexes/semantic-indexes/vector-indexes/> (visitato il 18/09/2026).
- [11] *Cypher Query Language Reference*. URL: <https://neo4j.com/docs/cypher-manual>.
- [12] *Gemini 3.5 Flash Model Card*. URL: <https://storage.googleapis.com/deepmind-media/Model-Cards/Gemini-3-5-Flash-Model-Card.pdf>.
- [13] Google. *Gemini 3.5 Flash*. 2026. URL: <https://ai.google.dev/gemini-api/docs/models/gemini-3.5-flash> (visitato il 18/09/2026).

- [14] Google DeepMind. *EmbeddingGemma Model Card*. 2025. URL: <https://huggingface.co/google/embeddinggemma-300m> (visitato il 18/09/2026).
- [15] Sedigheh Eslami et al. *Diffusion-Pretrained Dense and Contextual Embeddings*. 2026. DOI: 10.48550/arXiv.2602.11151. arXiv: 2602.11151 [cs.IR]. URL: <https://arxiv.org/abs/2602.11151>.
- [16] Michael Günther et al. *Late Chunking: Contextual Chunk Embeddings Using Long-Context Embedding Models*. 2024. arXiv: 2409.04701 [cs.IR]. URL: <https://arxiv.org/abs/2409.04701>.
- [17] Repubblica Italiana. *Decreto Legislativo 7 marzo 2005, n. 82: Codice dell'Amministrazione Digitale, art. 40*. 2005. URL: <https://www.normattiva.it/uri-res/N2Ls?urn:nir:stato:decreto.legislativo:2005-03-07;82~art40> (visitato il 18/09/2026).
- [18] Agenzia per l'Italia Digitale. *Linee guida sulla formazione, gestione e conservazione dei documenti informatici*. 2021. URL: https://www.agid.gov.it/sites/agid/files/2024-05/linee_guida_sul_documento_informatico.pdf (visitato il 18/09/2026).
- [19] Agenzia per l'Italia Digitale. *Allegato 2: Formati di file e riversamento*. 2021. URL: https://www.agid.gov.it/sites/agid/files/2024-06/allegato_2_formati_di_file_e_riversamento.pdf (visitato il 18/09/2026).